



Project Report on

CampusBazar

Student Marketplace Platform

Submitted in partial fulfillment of the requirement for the award of
degree in **Bachelor of Computer Applications** during the
Academic year 2023-26

Submitted by

1) U03AI23S0017 BHARATHI.O

Under the guidance of

Mrs. Amrutha BR

Acharya Bangalore B-School

Andrahalli Main Road, Off Magadi Road,
Bengaluru - 560091, Karnataka, India



Acharya Bangalore B-School
Department of Computer Science - BCA

Certificate

This is to certify that **BHARATHI.O** bearing Reg. No **U03AI23S0017** has satisfactorily completed the project entitled **CampusBazar** under the guidance and supervision of **Mrs. Amrutha BR** in partial fulfillment of VI Semester BCA course as prescribed by ABBS(Autonomous), Bangalore in our college during the academic year 2025-26.

Project Guide

(Mrs. Amrutha BR)

Head of the Department

(Mr.PraveenKumar V)

Examiners

1.....

2.....

Acknowledgement

I articulate my sincere gratitude to all those who helped me in making this venture a grand success, without whose constructive criticism as well as words of inspiration this project of mine would not have seen the light.

I take this opportunity to thank **Dr. Rajesh C**, Principal, Acharya Bangalore B-School, for permitting me to undertake this project as part of my curriculum.

I would like to mention my sincere gratitude to **Mr. Praveen Kumar V**, Head of the Department, BCA, for the constructive criticism and guidance and for the encouragement that has been a source of inspiration for me.

I would like to express my gratitude to my internal guide **Mrs. Amrutha BR**, Department of Computer Science for giving me the valuable information and advices that helped me a lot for the successful completion of my project work.

Last but not the least, my thanks goes to my parents, GOD and friends whose support and enthusiasm helped me greatly to make the project work a memorable event.

Bharathi.O

U03AI23S0017

Abstract

CampusBazar is a secure, localized digital marketplace explicitly engineered to facilitate peer-to-peer commerce among college students. Currently, students rely on inefficient, fragmented, and unsafe methods like physical notice boards, disorganized social media groups, or public classifieds to buy and sell academic materials, electronics, and daily essentials. These methods expose students to privacy violations, financial scams, and physical safety risks. To resolve these critical issues, CampusBazar provides a centralized, closed-loop web application restricted to users with verified institutional email addresses. Developed utilizing the modern MERN stack (MongoDB, Express.js, React.js, Node.js), the platform features dynamic role-based dashboards, advanced algorithmic product search, and integrated real-time WebSocket communication for private price negotiations. By eliminating complex shipping logistics and third-party payment gateways, CampusBazar fosters a sustainable, eco-friendly culture of reusing goods while drastically reducing the financial burden of higher education, ensuring maximum economic benefit and safety for the student community.

TABLE OF CONTENTS

.....

1.0 INTRODUCTION.....	7-13
1.1 Introduction	
1.2 Statement of Problem	
1.3 Objective of the Study	
1.4 Scope of the Study	
1.5 Feasibility Study	
1.5.1 Technical Feasibility	
1.5.2 Operational Feasibility	
1.5.3 Financial Feasibility	
2.0 SYSTEM ANALYSIS AND METHODOLOGY.....	14-24
2.1 Existing System.	
2.2 Limitations of Existing System	
2.3 Proposed System	
2.4 Advantages of Proposed System	
2.5 Problem Formulation	
2.6 System Analysis	
2.6.1 Exhaustive Functional Requirements	
2.6.2 System Functionalities and Architecture Modules	
2.6.3 Non-Functional Requirements	
2.7 Methodology	
2.7.1 Theoretical Introduction to Agile Methodology	
2.7.2 Rationale for Selecting Agile for CampusBazaar	

2.7.3 Execution of Agile Principles	
2.7.4 Sprint Planning and Ceremonial Execution	
2.7.5 Continuous Integration and Testing (The Testing Sprint)	
2.7.6 Deployment Strategy and Continuous Feedback	
3.0 SYSTEM DESIGN AND ARCHITECTURE.....	25-52
3.1 Data Flow Diagram (DFD)	
3.1.1 Level 0 DFD (Context Diagram)	
3.1.2 Level 1 DFD	
3.2 E-R Diagram	
3.3 Structured Chart	
3.4 Database Design	
3.5 Use Case Diagram	
3.6 Exhaustive Data Dictionary (Database Collections)	
3.6.1 Users Collection Schema	
3.6.2 Products Collection Schema	
3.6.3 Messages Collection Schema	
3.6.4 Wishlists Collection Schema	
3.7 Theoretical Coding Standards and Software Architecture	
3.8 CampusBazaar Full-Stack System Architecture	
3.9 Coding	
3.9.1 Sample Source Code (Main React Router Setup)	
3.9.2 Database (MongoDB Connection)	
3.9.3 Authentication (Login Route)	
3.9.4 Dashboard Login (Frontend UI)	
3.9.5 Messages (Socket.io & Model)	

3.9.6 Product & Seller Data	
3.9.7 Switch Role UI (Frontend)	
4.0 SYSTEM IMPLEMENTATION AND TESTING.....	53-61
4.1 System Implementation	
4.2 Hardware and Software Environment	
4.3 Testing Methodologies	
4.3.1 Unit Testing	
4.3.2 Integration Testing	
4.3.3 System Testing	
4.3.4 User Acceptance Testing	
4.4 Test Cases	
5.0 RESULTS AND SYSTEM SNAPSHOTS.....	62-83
5.1 Registration and Authentication Modules	
5.2 Buyer and Seller Dashboards	
5.3 Product Listing and Global Search	
5.4 Real-Time Chat Interface	
5.5 Admin Control Panel	
6.0 CONCLUSION AND FUTURE ENHANCEMENTS.....	84-88
6.1 Conclusion	
6.2 Future Enhancements	
6.3 BIBLIOGRAPHY / REFERENCES	

Chapter -1

Introduction

1.1 INTRODUCTION

The rapid evolution of digital platforms, cloud computing, and mobile-first web applications has fundamentally revolutionized the landscape of modern commerce. Global consumer behavior has irrevocably shifted from traditional, physical brick-and-mortar stores to dynamic, internet-based marketplaces. Platforms such as Amazon, eBay, and Facebook Marketplace have demonstrated the immense power of connecting buyers and sellers across vast geographical distances. However, while these generalized e-commerce giants and broad classified advertisement websites serve the global public adequately, specific, highly localized communities often remain drastically underserved by these generic solutions.

College and university campuses represent one such unique, closed, and highly dense economic ecosystem. Every single academic year, thousands of students experience a recurring, almost cyclical need to buy and sell highly specific goods: academic materials (like specialized textbooks and engineering calculators), electronics (laptops, tablets), dorm furniture (mini-fridges, desks, lamps), and daily living essentials.

CampusBazaar is proposed and developed as a dedicated, secure, and highly centralized web-based marketplace engineered specifically to facilitate peer-to-peer commerce among students within a specific college or university campus. By acting as an intelligent digital intermediary, CampusBazaar connects student Buyers and Sellers in a verified, trusted, and localized environment. This specialized platform entirely eliminates the need for complex shipping logistics, expensive delivery tracking, and third-party payment gateway fees. Simultaneously, it fosters a sustainable, eco-friendly culture of reusing goods, thereby reducing the carbon footprint and electronic waste generated by the student body.

1.1 STATEMENT OF PROBLEM

Currently, university students face significant friction, financial loss, and severe safety risks when attempting to buy or sell second-hand items. The existing methods for student commerce are highly fragmented, archaic, and deeply inefficient. The primary methods utilized today include:

1. **Physical Notice Boards:** Students print out flyers and pin them to corkboards. These are easily cluttered, destroyed by weather or vandalism, geographically restrictive (only visible if you walk past them), and lack any digital search capability.
2. **Social Media Group Chats:** Students use massive WhatsApp or Telegram groups. In these groups, messages disappear rapidly under the sheer volume of daily texts, making it impossible to catalog items or filter for specific needs.
3. **Public Internet Marketplaces:** Utilizing platforms like Craigslist or OLX exposes young, vulnerable students to "stranger danger," potential financial scams, and the physical risks associated with meeting completely unknown individuals off-campus to exchange cash. Furthermore, purchasing brand-new items from official, monopolistic campus bookstores or retail chains is often financially devastating for students operating on strict, limited budgets. There is a critical, glaring absence of a safe, categorized, and private digital platform restricted exclusively to the verified campus community.

1.2 OBJECTIVE OF THE STUDY

The primary objectives driving the research, design, and development of the CampusBazaar project are multifaceted, aiming to solve technological, economic, and social challenges simultaneously:

1. **Robust Technological Infrastructure:** To design, develop, and deploy a secure, high-performance, and responsive web application utilizing the modern MERN stack (MongoDB, Express.js, React.js, Node.js).
2. **Dynamic Role-Based Interfaces:** To provide highly intuitive, distinct, and personalized dashboards allowing students to act seamlessly as either Buyers or Sellers, managing their respective tasks efficiently without needing separate accounts.
3. **Secure, Real-Time Communication:** To architect and implement an integrated, real-time messaging system utilizing WebSockets (Socket.io) that facilitates private price

negotiation and meetup arrangements, ensuring students never have to compromise their personal mobile phone numbers.

4. **Advanced Product Discovery:** To engineer a robust algorithmic search and dynamic filtering system, allowing users to easily locate specific products across multiple predefined categories (e.g., Engineering Books, Electronics, Dorm Furniture) based on price and condition.
5. **Comprehensive Administrative Oversight:** To build a secure administrative control panel for monitoring overall platform activity, resolving user-reported disputes, analyzing traffic data, and maintaining strict community safety guidelines.

1.4 SCOPE OF THE STUDY

The scope of this project encompasses the complete end-to-end software development lifecycle (SDLC) of the CampusBazar web application, from initial requirement gathering to final cloud deployment.

Included within the Scope:

- User registration and authentication via standard encrypted passwords and Google OAuth 2.0 integration.
- Comprehensive product listing management (Create, Read, Update, Delete - CRUD operations) with cloud-based image hosting.
- Advanced search algorithms with dynamic, client-side filtering.
- A user-specific, persistent wishlist feature.
- Live, bidirectional WebSocket chat with read receipts and timestamps.
- A dedicated Admin portal for data visualization and user moderation.

Excluded from the Scope: The platform operates on the fundamental premise that financial transactions and the physical exchange of goods will occur face-to-face on the campus premises. Therefore, complex features such as an integrated online payment gateway (credit card processing), shipping calculators, delivery tracking APIs, and taxation modules are explicitly

excluded from the current scope. This maintains regulatory simplicity, avoids banking liabilities, and perfectly suits the localized nature of a college campus.

1.5 FEASIBILITY STUDY

The feasibility study is a critical phase of the SDLC that assesses the practicality, viability, and potential risks of the proposed CampusBazaar platform to ensure successful deployment within the academic timeframe and budget.

1.5.1 Technical Feasibility

The project is determined to be highly technically feasible. It utilizes the industry-standard MERN stack, which is renowned globally for its immense scalability, non-blocking asynchronous performance (via the Node.js event loop), and robust open-source community support. The implementation of real-time chat via the Socket.io library and secure, stateless authentication via JWT (JSON Web Tokens) are well-documented, proven architectural paradigms in modern web development. The development team possesses the necessary technical acumen, hardware resources, and IDE tools to utilize these advanced web technologies effectively.

1.5.2 Operational Feasibility

Operationally, the platform is guaranteed to integrate smoothly into the daily routines of the student body. The problem it solves—affordable access to necessary academic goods—is a universally acknowledged pain point among students, practically guaranteeing a high rate of organic user adoption. The user interface (UI) is designed following modern UX heuristics to be highly intuitive, requiring absolutely no technical training or manuals for end-users. Furthermore, since complex delivery logistics and online payment disputes are intentionally bypassed, the day-to-day administrative overhead is incredibly minimal, relying primarily on automated analytics and a self-policing community reporting system.

1.5.3 Financial Feasibility

The project is exceptionally financially viable, particularly in its initial prototyping and launch phases. By heavily relying on open-source technologies (React, Node) and free-tier cloud hosting infrastructure, initial capital requirements are driven down to nearly zero. MongoDB Atlas provides generous free-tier cloud database hosting, while platforms like Render (for backend APIs) and Vercel (for frontend React hosting) offer highly capable free deployment pipelines. Future operational scaling costs, incurred only as network traffic grows significantly, can easily be offset through localized campus advertising, university student-union sponsorships, or a freemium model offering premium "highlighted" product listings for sellers.

Chapter -2

System Study

2.1 EXISTING SYSTEM

The existing system for student commerce is not a single, unified software entity, but rather a chaotic, disorganized collection of informal, disconnected, and highly manual channels. Currently, students seeking to sell an item must print paper flyers, manually cut out tear-away phone number strips with scissors, and physically tape them to dormitory walls, library boards, and cafeteria notice boards. This process is not only environmentally wasteful but also severely limits the geographic reach of the advertisement to a small radius of foot traffic.

Simultaneously, students rely heavily on massive, disorganized WhatsApp or Telegram groups specific to their graduating class or major. These social platforms were inherently designed for casual conversation, not structured e-commerce, lacking any foundational features like inventory tracking or secure payment gateways. Finally, a significant portion of students, lacking alternatives, resort to the traditional monopoly of the university bookstore, buying expensive new books and selling them back at deeply discounted, unfair rates at the end of the semester.

This lack of a standardized digital marketplace forces students to waste valuable time and resources navigating multiple fragmented channels just to conduct a simple transaction.

2.2 LIMITATIONS OF EXISTING SYSTEM

The limitations of the current ecosystem are severe, frustrating, and negatively impact the student experience on a daily basis. The absence of a dedicated technological solution exacerbates inefficiencies across multiple dimensions of campus life:

- **Lack of Organization and Searchability:** Physical paper flyers cannot be digitally searched. If a student is looking for a specific chemistry textbook, they must walk around the entire campus reading walls, hoping to stumble upon a relevant listing before it is torn down or fades. In massive group chats, messages disappear rapidly under the volume of daily conversations, making it impossible to filter for specific items. The absence of structured database querying means potential buyers cannot filter by price range, academic department, or item condition, leading to severe informational friction.

- **Severe Safety Concerns:** Utilizing public internet marketplaces like Craigslist or Facebook Marketplace exposes young, often inexperienced students to strangers off-campus, leading to well-documented risks of theft, fraud, and physical assault. There is no identity verification mechanism in these legacy systems to ensure the person arriving for the exchange is actually a fellow student. This fundamental lack of institutional accountability creates an environment where malicious actors can easily exploit the campus community.
- **Privacy Risks:** To facilitate a sale using flyers or WhatsApp, students are forced to publicize their private mobile phone numbers to thousands of strangers, opening themselves up to spam, harassment, and unwanted solicitation. Once a phone number is permanently recorded in a public group chat or pinned to a public bulletin board, it cannot be easily retracted. This widespread exposure directly violates fundamental data privacy principles, placing the personal information of the student body at continuous risk.
- **Financial Exploitation:** Selling back to institutional bookstores often yields only 10% to 15% of the item's original retail value. The bookstore then resells the used book at an 80% markup, causing massive financial loss for the student body. Similarly, without transparent pricing analytics, students selling goods peer-to-peer often undervalue their items or fall victim to aggressive low-balling in unregulated group chats.

2.3 PROPOSED SYSTEM

CampusBazaar completely replaces these chaotic, dangerous, and outdated manual methods with a sleek, centralized, and highly secure digital platform built exclusively for the university ecosystem. It requires users to register using university-issued email addresses (e.g., domains ending in .edu or specific college domains), instantly creating a closed-loop, safe environment authenticated at the institutional level. By leveraging Node.js and Express for a robust backend API, the platform guarantees high availability and rapid data processing. Once authenticated, users are presented with a highly visual, categorically organized dashboard built dynamically using React components. They can instantly search for items using keywords, view high-resolution image galleries uploaded by peers, read detailed product conditions, and click a single button to initiate a private, in-app WebSocket chat with the seller to negotiate terms safely. Furthermore,

CampusBazaar introduces dedicated user profiles, allowing individuals to track their active listings, monitor wishlists, and manage their transaction history efficiently in one consolidated hub.

2.4 ADVANTAGES OF PROPOSED SYSTEM

The transition from the existing, archaic system to CampusBazaar offers overwhelming, measurable advantages to the university ecosystem. By digitizing and securing the local commerce workflow, the platform delivers distinct improvements in usability, security, and financial equity:

- **Enhanced Safety and Absolute Trust:** Because every single user on the platform is verified via a university email, transactions remain strictly within the trusted student community. The fear of meeting dangerous strangers is eliminated, as platform moderation can tie every digital action to an identifiable academic account. Furthermore, the implementation of administrative moderation and user reporting mechanisms ensures that suspicious activity is swiftly handled, maintaining a secure digital environment.
- **Efficient Discovery:** Categorized listings and a powerful algorithmic search engine save students hours of scrolling through irrelevant chat messages. Items are found in seconds through optimized MongoDB queries that immediately filter results by title, category, and price range. The integration of high-quality image hosting capabilities provides buyers with a clear, accurate representation of the product before committing to a negotiation, vastly improving the purchasing experience.
- **Total Privacy Maintained:** The built-in Socket.io messaging module completely eliminates the need to share personal contact details or social media profiles. If a negotiation fails, the user simply closes the chat, and the other party has no way to contact them outside the app. All communications are securely routed through the application server, protecting the mobile phone numbers and private emails of all participants from unauthorized extraction.
- **Maximum Economic Benefit:** By cutting out the middleman (the campus bookstore), sellers make significantly more money, and buyers pay significantly less. 100% of the negotiated price stays within the student economy, fostering a micro-economy that mutually benefits both parties. This peer-to-peer model drastically reduces the financial burden of higher education by making essential supplies like textbooks, electronics, and dorm furniture significantly more affordable.

2.5 PROBLEM FORMULATION

The entire software engineering endeavor is driven by the following formal, overarching problem statement: "How can modern software engineering principles, Agile methodologies, and full-stack web technologies be applied to architect and develop a highly scalable, secure, and usable digital marketplace that empowers college students to buy, sell, and trade second-hand goods locally, thereby directly mitigating the safety risks, privacy violations, and organizational inefficiencies inherent in current ad-hoc commerce methods?" Addressing this problem requires not only a robust backend infrastructure capable of handling concurrent data transactions but also an intuitive frontend interface that eliminates the steep learning curve traditionally associated with legacy institutional software.

2.6 SYSTEM ANALYSIS

2.6.1 Exhaustive Functional Requirements

Functional requirements define the specific, testable behaviors and functions the system must explicitly perform:

User (Buyer/Seller) Requirements:

- The system must allow users to register with a valid email and a strong password.
- The system must allow users to authenticate using Google OAuth 2.0.
- The system must allow authenticated users to switch contexts between a "Buyer Dashboard" and a "Seller Dashboard".
- The system must allow sellers to create a product listing consisting of a Title, Description, Price, Category, Condition, and up to 5 Image Uploads.
- The system must allow sellers to edit the details of their own active listings.
- The system must allow sellers to mark a listing as "Sold," which removes it from public search results.

- The system must allow buyers to search for products using a text-based keyword search bar.
- The system must allow buyers to filter search results by Category (e.g., Books, Electronics) and Price Range.
- The system must allow buyers to add available products to a personal "Wishlist" for future reference.
- The system must provide a real-time chat interface allowing a buyer to message a seller directly regarding a specific product.

Administrator Requirements:

- The system must provide a secure login portal specifically for administrators.
- The system must generate and display real-time analytics (Total Users, Active Listings, Messages Sent).
- The system must display a queue of user-submitted "Reports" flagging inappropriate content or abusive behavior.
- The system must grant administrators the power to permanently delete any product listing.
- The system must grant administrators the power to suspend or permanently ban user accounts.

2.6.2 System Functionalities and Architecture Modules

The architecture is divided into distinct, loosely coupled functional modules to ensure maintainability:

- **Authentication & Identity Module:** Handles password hashing (using bcrypt), JWT token generation, token verification middleware, and profile management.

- **Inventory & Catalog Module:** Manages the complex MongoDB aggregation pipelines required to fetch, sort, filter, and paginate grids of items efficiently. Handles interactions with Cloudinary for image blob storage.
- **Real-Time Communication Module:** Handles the Socket.io server-side event loops, managing dynamic chat rooms, routing messages to the correct socket IDs in real-time, and persisting the chat history to the NoSQL database for future retrieval.
- **Analytics & Moderation Module:** Provides data visualization, querying the database to generate charts for daily active users and total platform value, while handling the moderation queue.

2.6.3 Non-Functional Requirements

Non-functional requirements dictate the quality attributes, performance standards, and operational constraints of the system:

- **Performance Metrics:** The platform frontend must achieve a First Contentful Paint (FCP) of under 1.5 seconds on standard broadband. The backend Node.js server must handle routine API GET responses in under 200 milliseconds.
- **Scalability:** The backend must be completely stateless (storing session data in JWTs, not server memory), allowing for seamless horizontal scaling across multiple server instances or Docker containers during peak semester-start traffic spikes.
- **Usability & Accessibility:** The UI must be fully responsive, utilizing modern CSS Grid and Flexbox layouts to adapt flawlessly from 4K desktop monitors down to 320px mobile smartphone screens. It must adhere to WCAG 2.1 contrast standards to accommodate visually impaired students.

Strict Security Constraints: Passwords must be cryptographically hashed using bcrypt with a minimum salt round of 10. All API endpoints modifying data must be protected by Express middleware verifying the JWT signature to prevent unauthorized data mutation. Database connections must enforce TLS/SSL encryption

2.7 METHODOLOGY

2.7.1 Theoretical Introduction to Agile Methodology

The CampusBazaar project abandoned traditional, rigid, and sequential methodologies (such as the classical Waterfall model) in favor of being designed and developed based strictly on the Agile Software Development Life Cycle (SDLC). Agile is a highly iterative, time-boxed approach to software delivery that builds software incrementally from the very start of the project, rather than attempting to deliver a massive, finalized product all at once near the end of the timeline. Agile focuses on adaptive planning, evolutionary development, early delivery, and continual improvement.

2.7.2 Rationale for Selecting Agile for CampusBazaar

Agile was explicitly chosen because modern, full-stack web applications require immense flexibility and rapid adaptation to shifting user feedback, which rigid models simply cannot accommodate. The chosen MERN stack (MongoDB, Express.js, React.js, Node.js) is inherently highly modular, utilizing discrete React functional components on the frontend and separate Express router/controller files on the backend.

This granular modularity is perfectly suited for Agile's iterative cycles. It allowed the development team to adapt to UI/UX changes on the fly, dynamically tweak MongoDB NoSQL database schemas without breaking the entire application architecture (a common and fatal issue in rigid SQL databases), and integrate highly complex features—like the real-time WebSocket chat engine—gradually over several iterations without stalling overall progress.

2.7.3 Execution of Agile Principles

The project closely adhered to the core tenets of the Agile Manifesto throughout its lifecycle:

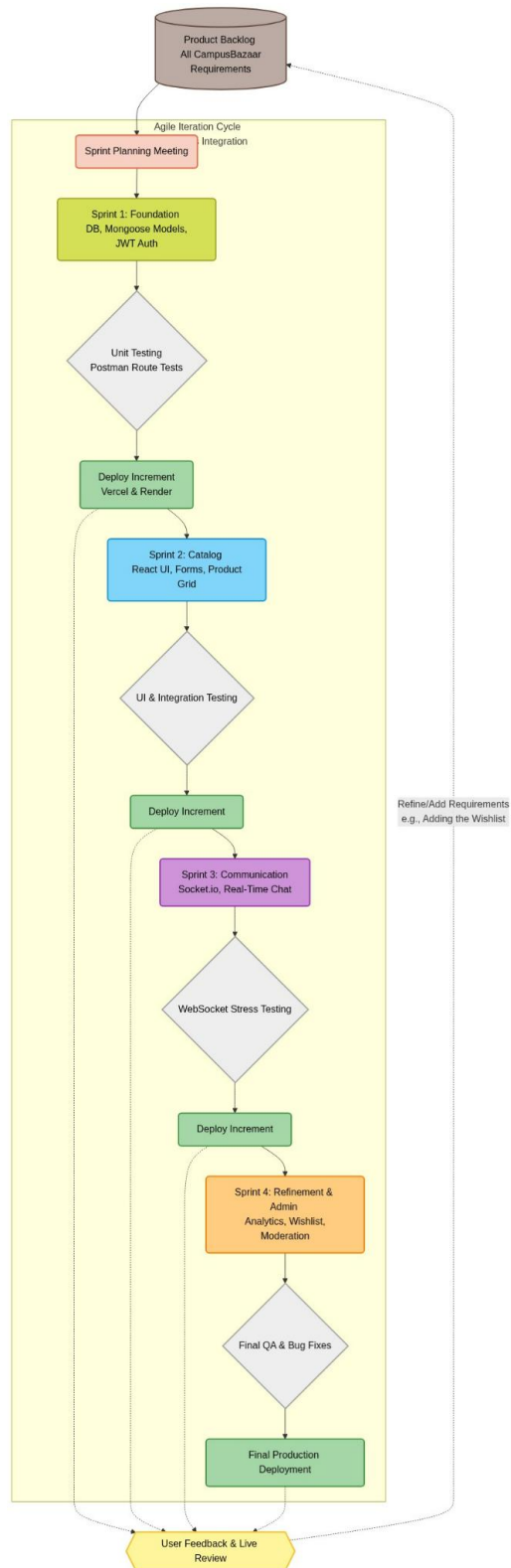
1. Highest priority is customer satisfaction through early and continuous delivery of valuable, working software modules.

2. Welcome changing requirements, even late in development, to provide a competitive advantage and a better final user experience.
3. Deliver working software frequently, utilizing strict, short sprint cycles (typically 2 weeks).
4. Continuous attention to technical excellence and good architectural design (enforcing the MVC pattern and DRY principles).

2.7.4 Sprint Planning and Ceremonial Execution

The project lifecycle was divided into strict 2-week iterations known as "Sprints." Before the commencement of each sprint, a Sprint Planning meeting was held to prioritize features from the master Product Backlog.

- Sprint 1 (Foundation): Focused purely on backend database connection logic, Mongoose user models, secure JWT authentication routes, and basic frontend routing.
- Sprint 2 (Catalog): Focused entirely on the frontend React UI, global state management, product creation forms, and building the dynamic product browsing grid.
- Sprint 3 (Communication): Dedicated exclusively to the complexities of implementing the Socket.io messaging system, ensuring instant delivery, and handling offline message persistence.
- Sprint 4 (Refinement & Admin): Focused on building the Admin dashboard, generating analytics, fixing UI bugs discovered during early testing, and finalizing deployment pipelines.

Figure 2.7.4: Agile Software Development Model

2.7.5 Continuous Integration and Testing (The Testing Sprint)

In the Agile model, testing is continuous and pervasive; it is not relegated to a single, stressful phase at the end of the development lifecycle. Instead of a massive testing bottleneck at the end of the semester, each sprint included its own rigorous unit, integration, and manual testing protocols. For example, when the complex product upload API was constructed in Sprint 2, it was fully tested using Postman—bombarding it with valid JSON payloads, malformed data, and oversized image files—and fully validated to handle edge cases gracefully before Sprint 2 was officially considered "Done." This ensured that bugs were caught immediately at the source rather than compounding into larger architectural failures later in the project.

2.7.6 Deployment Strategy and Continuous Feedback

Working increments of the software were deployed to live cloud hosting platforms (Vercel for the React frontend, Render for the Node.js backend) at the conclusion of major sprints. This CI/CD (Continuous Integration/Continuous Deployment) approach ensured the application was always in a live, usable, and remotely accessible state. By deploying early and often, the team could interact with the live software on actual mobile devices across different network conditions, gather immediate feedback regarding the user interface, and make necessary incremental improvements. For example, the highly requested "Wishlist" feature was dynamically added to the product backlog in Sprint 4 after realizing during a live peer-review session that users wanted a specific way to save items before committing to a direct chat with the seller—a pivot that would have been impossible under a traditional Waterfall methodology.

Chapter -3

System Design and Development

System design is the process of defining the architecture, modules, interfaces, and data for a system to satisfy specified requirements. Visual modeling is crucial for understanding complex systems before writing actual code, ensuring that all functional and non-functional objectives are met seamlessly.

3.1 Data Flow Diagram (DFD)

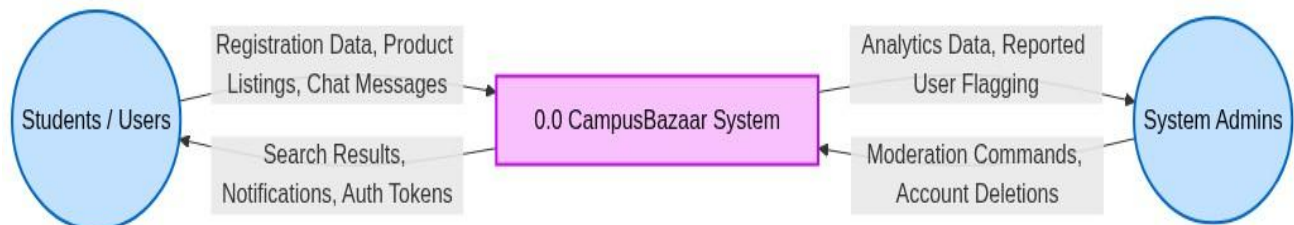
A Data Flow Diagram (DFD) maps out the flow of information for any process or system. It uses defined symbols like rectangles, circles, and arrows to show data inputs, outputs, storage points, and the routes between each destination. By acting as a universal visual language, DFDs bridge the communication gap between technical developers and non-technical stakeholders.

Furthermore, mapping out these pathways during the early design phases ensures that no "data black holes" exist—guaranteeing that every piece of user input is properly processed, stored, or outputted by the application.

3.1.1 Level 0 DFD (Context Diagram)

The Level 0 DFD, also known as a context diagram, illustrates the entire CampusBazaar system as a single high-level process, showing how external entities (Users and Administrators) interact with the platform at a macro level.

Figure 3.1.1: Data Flow Diagram



3.1.2 Level 1 DFD

The Level 1 DFD breaks down the main system into its primary sub-processes, showing the granular flow of data between the authentication, product management, search, and messaging engines.

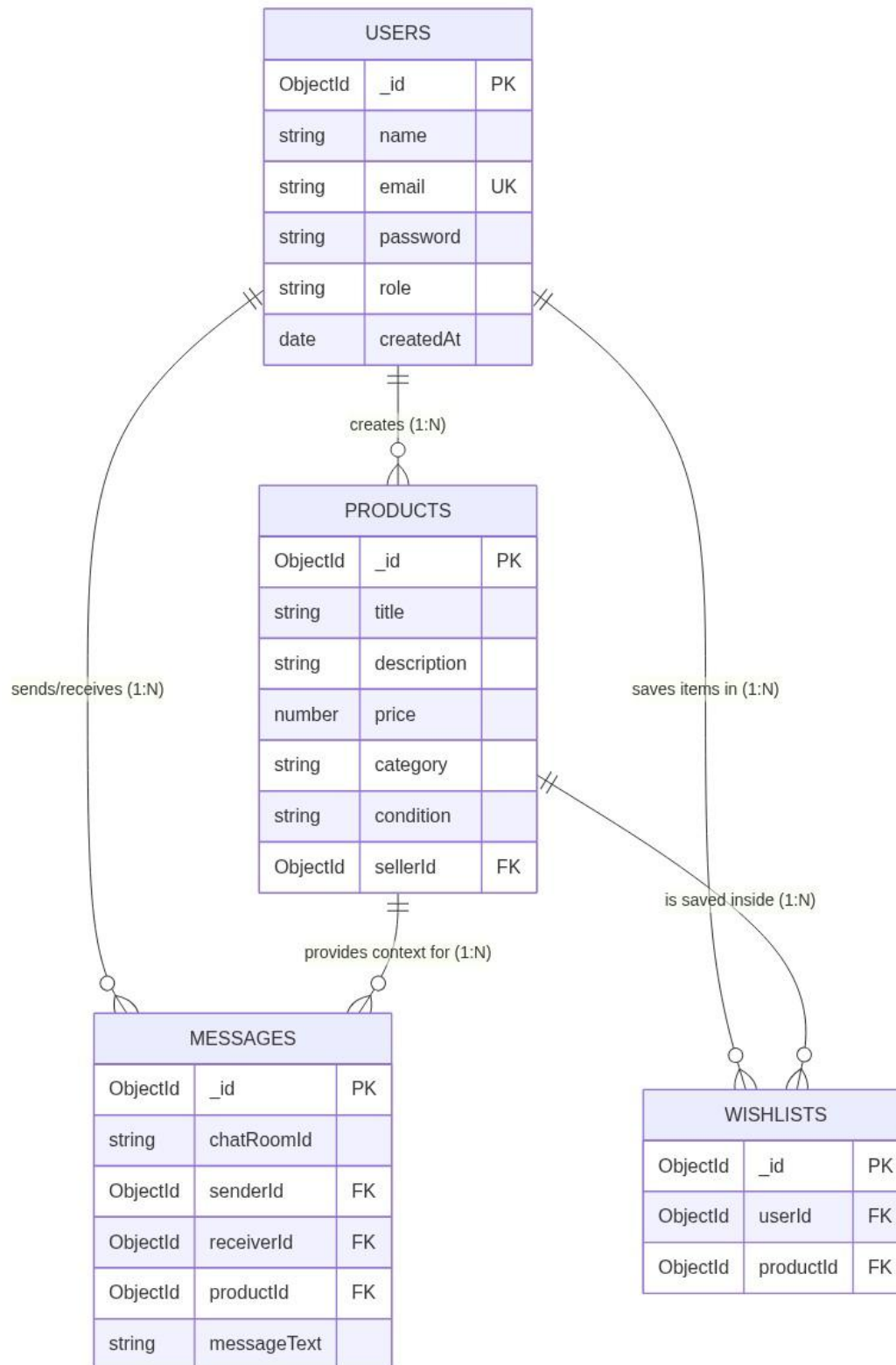
Figure 3.1.2: Data Flow Diagram



3.2 E-R DIAGRAM

An Entity-Relationship (ER) Diagram is a structural diagram for use in database design. It visualizes the relational connections between the NoSQL document collections, highlighting the cardinality of how Users interact with Products and Messages within the CampusBazaar ecosystem. Unlike traditional SQL databases, this NoSQL ER diagram specifically helps the engineering team visualize where document referencing should be used versus document embedding. Understanding these relationship cardinalities is critical for writing efficient MongoDB aggregation pipelines that can join collections (via \$lookup) without causing performance bottlenecks.

- **Users & Products (One-to-Many & Many-to-Many):** A single User (acting as a seller) can list multiple Products, represented by a `userId` reference inside the Product document. Conversely, the User collection utilizes array-based document referencing to track wishlist and recentlyViewed items (arrays of Product ObjectIds). This prevents data duplication and keeps the User document size well within MongoDB's 16MB limit.
- **Messaging Subsystem (Many-to-Many resolved via reference collections):** To handle real-time chat between buyers and sellers, the system uses intermediate collections. A Conversation document links a `buyerId`, `sellerId`, and a specific `productId`. Individual Message documents then reference the parent `conversationId`, alongside the `senderId` and `receiverId`. Embedding thousands of messages directly inside a Product or User document would cause severe performance degradation; therefore, the ER diagram dictates strict *document referencing* for messages.
- **Transactions & Moderation (One-to-One / One-to-Many):** Purchase and Report entities act as relational junctions. A Purchase ties a `buyerId` to a `productId`, while a Report links a `reporterId` to a `productId`.

Figure 3.2: Entity Relationship Diagram

3.3 STRUCTURED CHART

Figure 3.3: Structured chart



3.4 DATABASE DESIGN

Database design plays an important role in the development of any information system. It determines how information is stored, organized, and retrieved efficiently. The CampusBazaar Digital Marketplace uses MongoDB as its primary database management system due to its inherent flexibility, massive scalability, and robust support for JSON-like, document-oriented storage.

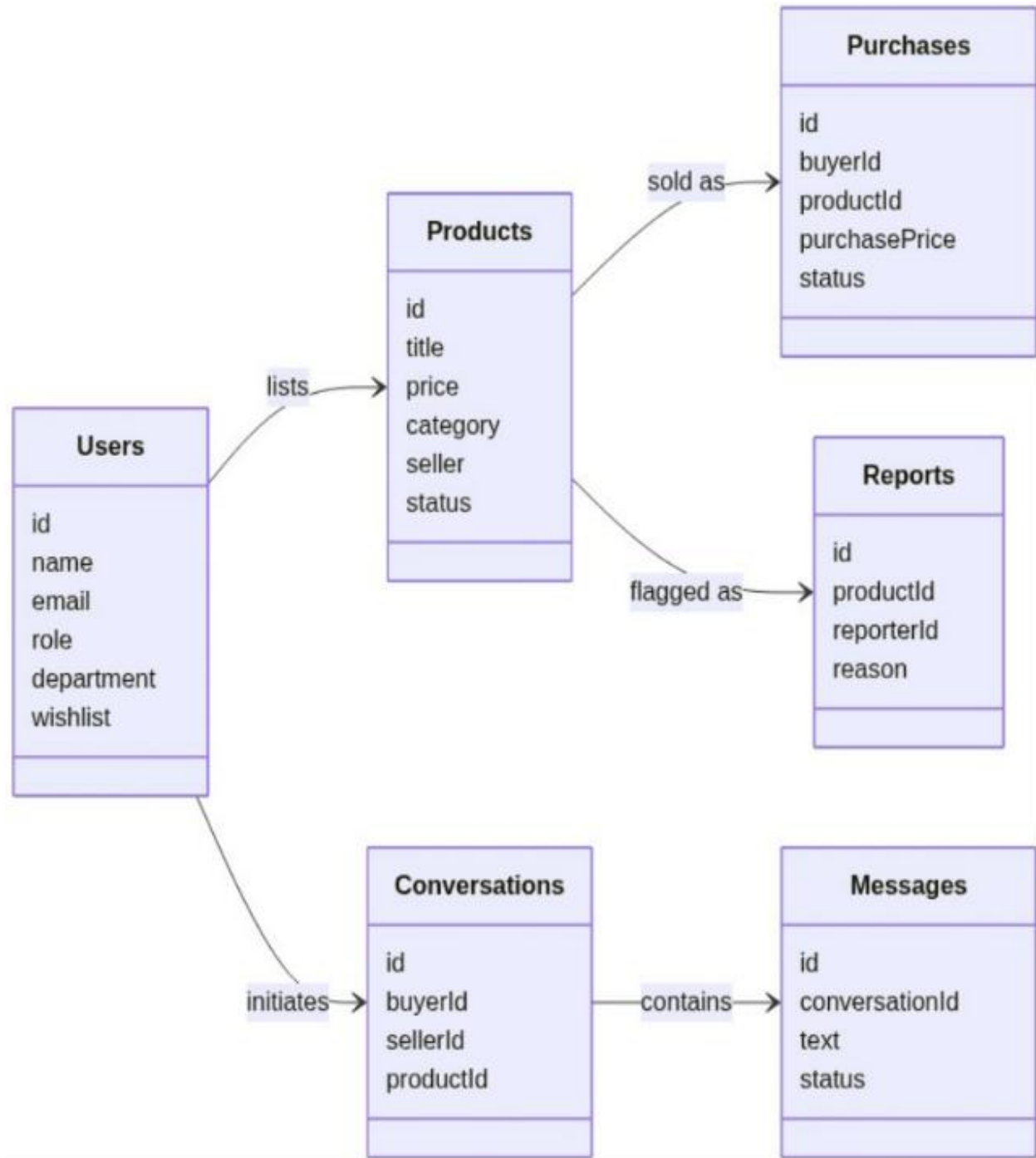
The database consists of multiple distinct collections that store information related to verified users, active product listings, peer-to-peer messages, and personalized wishlists. Each collection is meticulously designed to support a specific functional module of the e-commerce application.

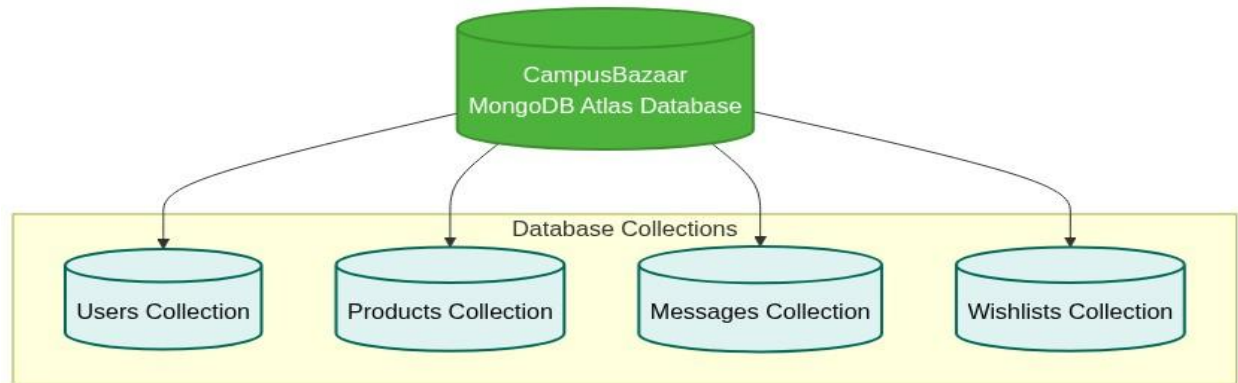
The Users collection stores authentication details, encrypted passwords, and profile information of verified student members and administrators. The Products collection maintains comprehensive information related to items listed for sale, including dynamic pricing, searchable categories, conditions, and references to cloud-hosted image galleries. The Messages collection stores the real-time chat history between buyers and sellers to ensure offline communication continuity. An additional Wishlists collection manages the specific items that users have saved for future reference.

This strategic NoSQL database design ensures high data consistency, rapid data retrieval during complex text-based search queries, and immense horizontal scalability to handle future traffic spikes during the start of academic semesters.

- Core Collections:
 1. Users (users): Authenticates credentials, stores roles (buyer, seller, admin), and manages wishlists.
 2. Products (products): Manages item listings, prices, categories, conditions, and embeds seller rating details.
 3. Conversations (conversations): Tracks real-time chat threads between buyers and sellers regarding specific products.
 4. Messages (messages): Stores individual real-time chat messages and their read/unread status.
 5. Purchases (purchases): Logs completed marketplace transactions, historical pricing, and fulfillment status.
 6. Reports (reports): Tracks flagged items, moderation reasons, and admin resolution status.

Figure 3.4.1: Database Design Diagram

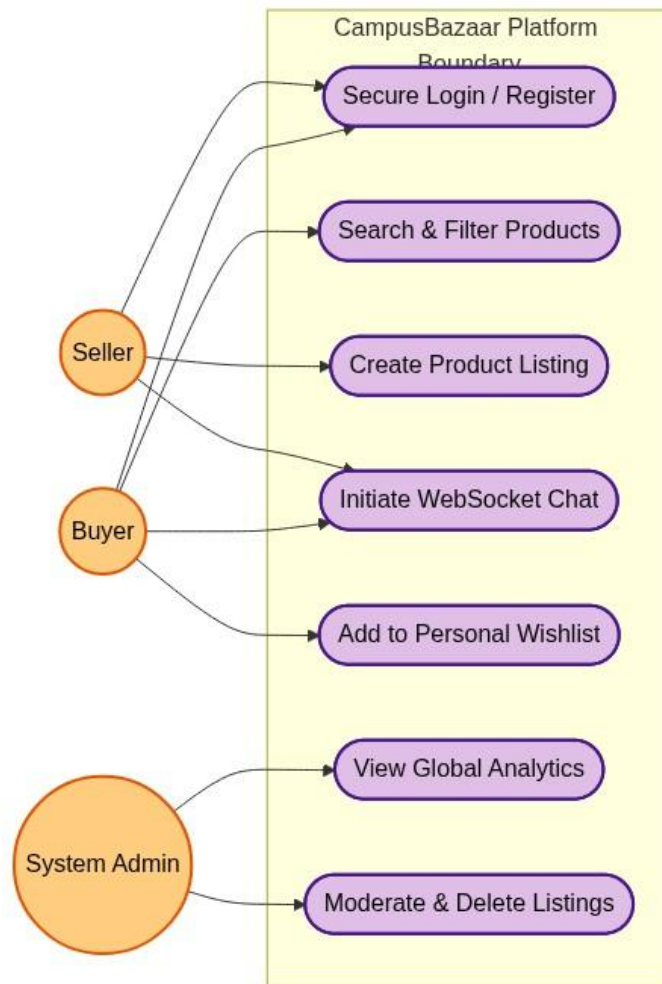
**Figure 3.4.2: Database Collections Diagram**



3.5 Use Case Diagram

The Use Case Diagram defines the behavioral interactions between the primary actors (Buyer, Seller, Admin) and the specific functional features available within the system boundary. It outlines who can do what. Clearly defining these system boundaries prevents "feature creep" during the Agile development cycles. Furthermore, this diagram directly informs the Quality Assurance (QA) testing phase, allowing developers to formulate precise unit tests based directly on the explicit interactions mapped for each actor.

Figure 3.5: User Case Diagram



3.6 Exhaustive Data Dictionary (Database Collections)

A data dictionary contains detailed metadata about data elements, defining the structure, types, and constraints of the database schemas. This dictionary acts as the single source of truth for the backend engineering team when writing Mongoose schema validations. By enforcing strict data types and constraints here, the system mathematically prevents malformed API payloads from crashing the database or causing downstream UI rendering errors.

3.6.1 Users Collection Schema

This collection stores all data related to authenticated individuals on the platform.

Figure 3.6.1: User Collection Schema

Field Name	Data Type	Constraint	Description
<code>_id</code>	ObjectId	Primary Key	Unique 24-character hex string auto-generated by MongoDB.
<code>name</code>	String	Required	Full name of the student. Displayed on their public seller profile.
<code>email</code>	String	Required, Unique	Institutional email used for login and verification. Indexed for speed.
<code>password</code>	String	Required	Bcrypt hashed string. Never stored in plain text.
<code>role</code>	String	Enum	Determines system privileges. Options: ['user', 'admin'].
<code>profilePicture</code>	String	Optional	URL referencing a secure Cloudinary image blob.
<code>createdAt</code>	Date	Auto-generated	Exact timestamp of account creation for analytical purposes.

3.6.2 products Collection Schema

This collection stores all details regarding items listed for sale.

Figure 3.6.2: Products Collection Schema

Field Name	Data Type	Constraint	Description
<code>_id</code>	ObjectId	Primary Key	Unique identifier for the listing.
<code>title</code>	String	Required, Indexed	The name of the item. Indexed for fast text search algorithms.
<code>description</code>	String	Required, > 20 chars	Detailed explanation of the item, allowing buyers to read specifics.
<code>price</code>	Number	Required, Min: 0	Listed price in local currency. Cannot be a negative integer.
<code>category</code>	String	Enum	Maps to UI filters. Options: 'Books', 'Furniture', 'Electronics', 'Other'.
<code>condition</code>	String	Enum	Options: 'New', 'Like New', 'Good', 'Used', 'Refurbished'.
<code>images</code>	Array[String]	Max length: 5	Array of Cloudinary image URLs to render the product gallery.

sellerId	ObjectId	Foreign Key	References the <code>_id</code> in the users collection. Establishes ownership.
status	String	Enum	Options: 'Available', 'Pending Negotiation', 'Sold'.

3.6.3 Messages Collection Schema

This collection persists the chat history between users to ensure reliable offline communication.

Figure 3.6.3: Message Collection Schema

Field Name	Data Type	Constraint	Description
<code>_id</code>	ObjectId	Primary Key	Unique identifier for the specific message bubble.
chatRoomId	String	Required, Indexed	Concatenated string of sender and receiver IDs to group chats together.
senderId	ObjectId	Foreign Key	References the users collection (the person typing).
receiverId	ObjectId	Foreign Key	References the users collection (the person receiving).
productId	ObjectId	Foreign Key	References the products collection to establish context for the chat.

messageText	String	Required	The actual text string of the chat message.
readStatus	Boolean	Default: false	Used to trigger blue-tick UI notifications when true.
timestamp	Date	Auto-generated	Exact time the message was sent to the server.

3.6.4 Wishlists Collection Schema

This collection tracks user interest without requiring them to commit to a direct chat with the seller.

Figure 3.6.4: Message Collection Schema

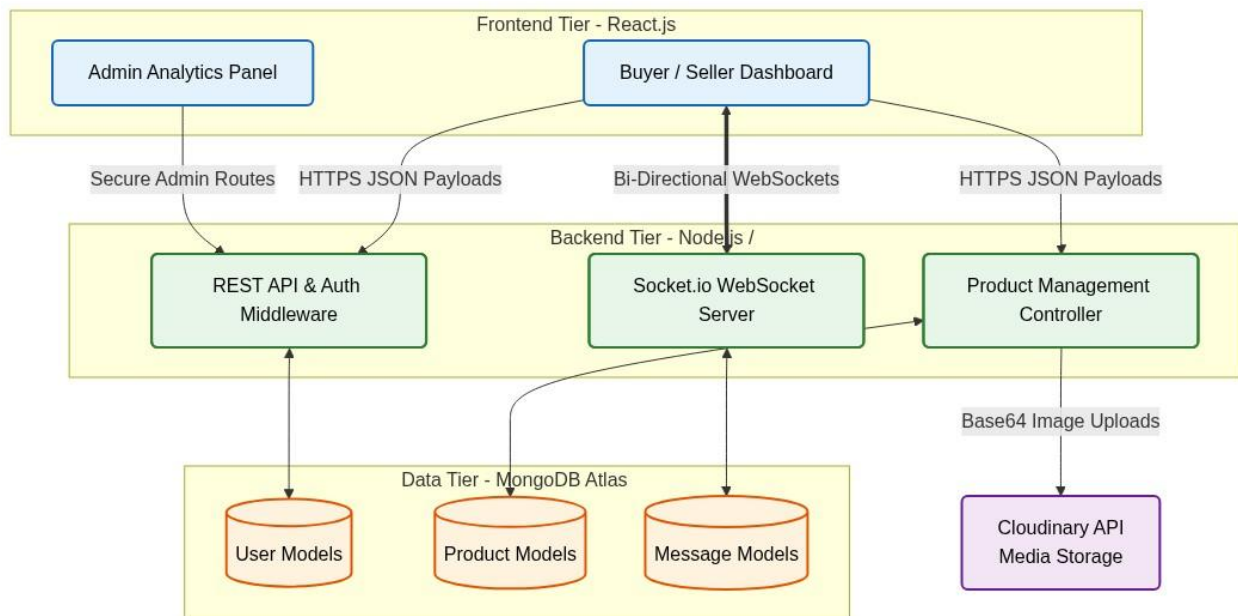
Field Name	Data Type	Constraint	Description
_id	ObjectId	Primary Key	Unique identifier auto-generated by MongoDB.
userId	ObjectId	Foreign Key	The buyer who saved the item.
productId	ObjectId	Foreign Key	The specific item being saved.
addedAt	Date	Auto-generated	Timestamp of when the item was saved.

3.7 Theoretical Coding Standards and Software Architecture

Adhering to professional coding standards is paramount for ensuring the codebase remains clean, readable, and scalable over time, especially when handing the project off to future maintainers. Consistent coding standards drastically reduce the cognitive load required for new developers onboarding onto the project. By employing automated tools like ESLint and Prettier alongside strict pull-request reviews, the engineering team guarantees that these architectural decisions remain intact over the long term.

- **Strict MVC Architecture:** The Node.js/Express backend strictly follows the Model-View-Controller design pattern. Database schemas are completely isolated in the `models/` directory using Mongoose. The core business logic is handled exclusively in the `controllers/` directory. The API endpoint routes are mapped cleanly in the `routes/` directory, acting solely as traffic directors.
- **Component Reusability (React):** The frontend user interface is built using React.js, which enforces a highly modular, component-based structure. Complex UI elements, such as a `<ProductCard />`, `<Navbar />`, or a generic `<CustomButton />`, are designed as isolated, reusable functions. This strictly adheres to the DRY (Don't Repeat Yourself) principle, significantly reducing codebase size.
- **Security through Environment Variables:** All sensitive configuration data—including MongoDB Connection URIs, JWT cryptographic Secret Keys, and third-party Cloudinary API keys—are stored securely in local `.env` files. These files are strictly excluded from version control (using `.gitignore`) to prevent catastrophic security breaches.
- **RESTful API Design Paradigm:** All backend API endpoints strictly follow standard REST naming conventions and utilize appropriate HTTP verbs. For example, a `GET /api/products` request fetches all products, while a `POST /api/products` request creates a new one. Appropriate HTTP status codes (200, 201, 400, 401, 500) are rigorously applied to ensure the frontend client handles errors gracefully.

3.8 CampusBazaar Full-Stack System Architecture

Figure 3.8.1: Full-Stack System Architecture

3.9 CODING

The coding phase involves translating the theoretical system design into a functional, deployable software application. The CampusBazaar Digital Marketplace was developed from the ground up using the highly scalable MERN Stack, consisting of MongoDB, Express.js, React.js, and Node.js. The frontend was developed using React.js and modern CSS practices to create a highly responsive, mobile-first, and interactive user interface that feels like a native application. The backend was implemented using Node.js and Express.js to securely handle REST API requests, complex business logic, JSON Web Token (JWT) authentication, and database CRUD operations. MongoDB Atlas was utilized to store all application data efficiently in the cloud.

The project strictly follows a modular Model-View-Controller (MVC) and component-based architecture. Separate Mongoose models, Express controllers, API routes, and isolated React functional components are created for different specific features. This decoupled structure drastically improves the readability, maintainability, and horizontal scalability of the overall application.

Major coding modules implemented in the system include Secure User Authentication (with encrypted JWT sessions), Product Catalog Management (handling Cloudinary image uploads), Real-Time Messaging (utilizing Socket.io for live chat), interactive Buyer/Seller Dashboards, Wishlist Management, and a secure Administrative Analytics Portal. This highly modular coding approach allows massive future enhancements—such as integrating digital payment gateways—to be seamlessly added without breaking existing core functionalities.

The successful execution of these code modules resulted in a robust, fully functional digital marketplace that fundamentally improves student safety, financial equity, and commerce organization within the university campus.

3.9.1: Sample Source Code (Main React Router Setup - App.jsx)

```
import React from 'react';

import { BrowserRouter, Routes, Route } from 'react-router-dom';

import { AuthProvider } from './context/AuthContext';

import { WishlistProvider } from './context/WishlistContext';

import ProtectedRoute from './components/ProtectedRoute';

import Home from './pages/Home';

import Register from './pages/Register';

import Login from './pages/Login';

import ChooseRole from './pages/ChooseRole';

import BuyerDashboard from './pages/BuyerDashboard';

import SellerDashboard from './pages/SellerDashboard';

import AdminLogin from './pages/AdminLogin';

import AdminDashboard from './pages/AdminDashboard';
```

```
function App() {  
  return (  
    <BrowserRouter>  
      <AuthProvider>  
        <WishlistProvider>  
          <Routes>  
            { /* Public Routes */}  
            <Route path="/" element={<Home />} />  
            <Route path="/register" element={<Register />} />  
            <Route path="/login" element={<Login />} />  
  
            <Route path="/admin/login" element={<AdminLogin />} />  
            <Route path="/admin/dashboard" element={<AdminDashboard />} />  
  
            { /* Protected Routes */}  
            <Route element={<ProtectedRoute />}>  
              <Route path="/choose-role" element={<ChooseRole />} />  
            </Route>  
  
            { /* Role-Specific Protected Routes */}  
            <Route element={<ProtectedRoute requireRole="buyer" />}>  
              <Route path="/buyer/dashboard" element={<BuyerDashboard />} />  
            </Route>  
  
            <Route element={<ProtectedRoute requireRole="seller" />}>
```

```
      <Route path="/seller/dashboard" element={<SellerDashboard />} />
    </Route>
  </Routes>
</WishlistProvider>
</AuthProvider>
</BrowserRouter>
);
}
export default App;
```

3.9.2. Database (MongoDB Connection)

The database connection is set up in backend/server.js using Mongoose:

```
const mongoURI = process.env.MONGO_URI || 'mongodb://127.0.0.1:27017/campusbazar';

mongoose.connect(mongoURI)

.then(async () => {

  console.log('Connected to MongoDB successfully.');

await seedDatabase();



server.listen(PORT, () => {



console.log(`Backend server running on http://localhost:${PORT}`);



});



})



.catch((err) => {



console.error('Failed to connect to MongoDB.');


```

```
console.error(err);

server.listen(PORT, () => {

console.log(`Backend server running on http://localhost:${PORT} (Database disconnected)`);

});

});
```

3.9.3. Authentication (Login Route)

backend handles authentication in backend/routes/auth.js.

```
router.post('/login', async (req, res) => {

try {

const { email, password } = req.body;

const user = await User.findOne({ email });

if (!user) return res.status(401).json({ message: 'Invalid credentials' });

const isMatch = await bcrypt.compare(password, user.password);

if (!isMatch) {

return res.status(401).json({ message: 'Invalid credentials' });

}

user.loginAttempts = 0;

user.lockUntil = undefined;

await user.save();

res.json(user);
```

```
} catch (error) {  
  
res.status(500).json({ message: 'Internal server error' });  
  
}  
  
});
```

3.9.4 Dashboard Login (Frontend UI)

The login form is handled in src/pages/Login.jsx. It uses custom useAuth() hook to process the login and route the user to their respective role dashboard:

```
const handleSubmit = async (e) => {  
  
e.preventDefault();  
  
const res = await loginWithPassword(formData.email, formData.password,  
formData.rememberMe);  
  
if (res.success) {  
  
const fullUser = res.user;  
  
if (fullUser && fullUser.role && fullUser.role !== 'buyer' && fullUser.role !== 'seller') {  
  
navigate('/choose-role');  
  
} else if (fullUser && fullUser.role) {  
  
selectRole(fullUser.role); // Directs to Buyer/Seller dashboard based on role  
  
} else {  
  
navigate('/choose-role');  
  
}  
  
} else {
```

```
setGlobalError(res.message);  
  
}  
  
};
```

3.9.5. Messages (Socket.io & Model)

It is using Socket.io in backend/server.js for real-time messaging, backed by the Message schema from backend/models/Message.js:

```
const messageSchema = new mongoose.Schema({  
  
  conversationId: { type: mongoose.Schema.Types.ObjectId, ref: 'Conversation', required: true },  
  
  productId: { type: mongoose.Schema.Types.ObjectId, ref: 'Product', required: true },  
  
  senderId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },  
  
  receiverId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },  
  
  senderRole: { type: String, enum: ['buyer', 'seller'], required: true },  
  
  receiverRole: { type: String, enum: ['buyer', 'seller'], required: true },  
  
  text: { type: String, required: true },  
  
  status: { type: String, enum: ['sent', 'read'], default: 'sent' }  
  
}, { timestamps: true });  
  
socket.on('send_message', async (data) => {  
  
  io.to(`${receiverId}_${receiverRole}`).emit('receive_message', message);  
  
  io.to(`${senderId}_${senderRole}`).emit('message_sent', message);  
  
});
```

3.9.6. Product & Seller Data

The product and seller details are stored together in the Product schema located in backend/models/Product.js:

```
const productSchema = new mongoose.Schema({
  title: { type: String, required: true },
  price: { type: Number, required: true },
  category: { type: String, required: true },
  condition: { type: String, required: true },
  seller: {
    userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
    name: { type: String, required: true },
    rating: { type: Number, default: 0 },
    verified: { type: Boolean, default: false },
    listingsCount: { type: Number, default: 1 },
    responseRate: { type: String, default: '100%' }
  },
  status: { type: String, enum: ['Pending', 'Active', 'Sold', 'Rejected'], default: 'Active' },
});
```

3.9.7 Switch Role UI (Frontend)

When users log in without a defined role or decide to switch, they are taken to the ChooseRole.jsx page. It handles selecting between a "Buyer" and "Seller" by invoking the selectRole function from Auth context:

* Buyer Card *

```
<div className="group relative bg-white border border-slate-200 rounded-3xl p-8 hover:border-theme-maroon hover:shadow-2xl hover:shadow-theme-maroon/10 transition-all duration-300 flex flex-col h-full transform hover:-translate-y-1.5 cursor-pointer"
```

```
onClick={() => selectRole('buyer')}}>
```

```
<div className="w-16 h-16 bg-theme-maroon/10 rounded-2xl flex items-center justify-center mb-6 group-hover:bg-theme-maroon group-hover:text-white text-theme-maroon transition-colors duration-300">
```

```
<ShoppingCart className="w-8 h-8" />
```

```
</div>
```

```
<h2 className="text-2xl font-extrabold text-slate-900 mb-3">Buyer</h2>
```

```
<p className="text-base text-slate-500 mb-8 flex-grow">
```

Browse campus listings, save favorites, contact sellers, and purchase student essentials.

```
</p>
```

```
<button className="w-full bg-slate-50 border border-slate-100 text-slate-700 font-bold py-3.5 px-6 rounded-xl group-hover:bg-theme-maroon group-hover:border-theme-maroon group-hover:text-white transition-all duration-300 text-lg shadow-sm">
```

Continue as Buyer

```
</button>
```

```
</div>
```

* Seller Card *

```
<div className="group relative bg-white border border-slate-200 rounded-3xl p-8 hover:border-theme-dark-maroon hover:shadow-2xl hover:shadow-theme-dark-maroon/10 transition-all duration-300 flex flex-col h-full transform hover:-translate-y-1.5 cursor-pointer"
```

```
onClick={() => selectRole('seller')}}>
```

```
<div className="w-16 h-16 bg-theme-dark-maroon/10 rounded-2xl flex items-center justify-center mb-6 group-hover:bg-theme-dark-maroon group-hover:text-white text-theme-dark-maroon transition-colors duration-300">
```

```
<Store className="w-8 h-8" />
```

```
</div>
```

```
<h2 className="text-2xl font-extrabold text-slate-900 mb-3">Seller</h2>
```

... text and button ...

```
</div>
```

3.9.8. Backend: Marketplace Product Filtering

marketplace.js route has a complex query builder that handles searching, category filtering, condition filtering, and pagination.

```
router.get('/products', async (req, res) => {
```

```
  try {
```

```
    const { search, category, condition, sort, page = 1, limit = 12 } = req.query;
```

```
    let query = {};
```

```
    const categoryNames = ['Books', 'Calculators', 'Electronics', 'Bicycles', 'Lab Equipment', 'Hostel Essentials', 'Stationery', 'Furniture', 'Miscellaneous'];
```

```
    if (search) {
```

```
      const searchLower = search.toLowerCase().trim();
```

```
      const matchedCategory = categoryNames.find(c => {
```

```
const cLower = c.toLowerCase();

return cLower === searchLower || cLower === searchLower + 's' || cLower + 's' ===
searchLower;

});

if (matchedCategory) {

  query.category = matchedCategory;

} else {

  query.$or = [

    { title: { $regex: search, $options: 'i' } },

    { category: { $regex: search, $options: 'i' } },

    { description: { $regex: search, $options: 'i' } },

    { tags: { $regex: search, $options: 'i' } }

  ];

}

}

if (category && category !== 'All Categories') query.category = category;

if (condition && condition !== 'All Conditions') query.condition = condition;

let sortObj = {};

switch (sort) {

  case 'Price Low to High': sortObj.price = 1; break;

  case 'Price High to Low': sortObj.price = -1; break;
```

```
case 'Most Viewed': sortObj.views = -1; break;

case 'Most Wishlisted': sortObj.wishlistCount = -1; break;

case 'Oldest First': sortObj.createdAt = 1; break;

case 'Newest First':

default: sortObj.createdAt = -1; break;

}

const skip = (parseInt(page) - 1) * parseInt(limit);

const products = await Product.find(query)

.sort(sortObj)

.skip(skip)

.limit(parseInt(limit));
```

Chapter -4

System Implementation

4.1 Theoretical Overview of the Technology Stack (MERN)

The CampusBazaar platform was engineered using the MERN stack. This specific stack was chosen for its uniformity; the entire application, from the browser rendering the UI to the server querying the database, is written in a single language: JavaScript (and JSON).

- **MongoDB (The Database):** MongoDB is a NoSQL document-oriented database. Unlike traditional SQL databases that use rigid tables and rows, MongoDB stores data in flexible, JSON-like BSON documents. This flexibility was crucial for an e-commerce platform where product attributes can vary wildly (e.g., a laptop has RAM specifications, a textbook has an ISBN). MongoDB Atlas was utilized to host the database entirely in the cloud.
- **Express.js (The Backend Framework):** Express is a minimal, fast, and unopinionated web framework for Node.js. It provides robust routing, allowing the server to handle complex HTTP GET, POST, PUT, and DELETE requests efficiently. It also allows the implementation of middleware, which intercepts requests to verify security tokens before allowing access to the database.
- **React.js (The Frontend Library):** Developed by Meta (Facebook), React is a declarative, efficient, and flexible JavaScript library for building user interfaces. It utilizes a Virtual DOM (Document Object Model) to update only the specific parts of the screen that have changed (like a chat message popping up), rather than reloading the entire webpage. This provides a lightning-fast, "app-like" experience.
- **Node.js (The Runtime Environment):** Node.js allows JavaScript to be run on the server, outside of a web browser. It uses an event-driven, non-blocking I/O model that makes it incredibly lightweight and efficient, perfect for data-intensive real-time applications like the WebSocket chat engine used in CampusBazaar.

4.2 HARDWARE AND SOFTWARE REQUIREMENTS

The implementation of the CampusBazaar Digital Marketplace requires specific hardware and software resources for development, local testing, and final cloud execution. The system has

been developed using modern web technologies and can run efficiently on standard computing devices available to students in educational institutions.

Hardware Requirements

- System: Personal Computer or Laptop
- Processor: Intel Core i5 / AMD Ryzen 5 or Above
- RAM: 8 GB Minimum
- Monitor: 15-inch LED Monitor (or standard laptop display)
- Keyboard: Standard Keyboard
- Mouse: Optical Mouse or Trackpad
- Hard Disk Capacity: 256 GB SSD (Solid State Drive)

Software Requirements

- Operating System: Windows 11
- Front End Framework: React.js, Tailwind CSS, Vite
- Back End Environment: Node.js, Express.js, Socket.io
- Database Management: MongoDB Atlas (Cloud NoSQL Database)
- Web Browser: Google Chrome / Microsoft Edge / Safari
- IDE (Integrated Development Environment): Visual Studio Code

These requirements are sufficient for local development and execution. Once cloud-deployed, end-users can seamlessly access the platform via any modern mobile or desktop web browser.

4.2.1 About the Software

The CampusBazaar Digital Marketplace is built entirely on the modern MERN Stack (MongoDB, Express.js, React.js, Node.js), adhering to a strict MVC architecture for maximum maintainability and scalability.

The frontend utilizes React.js and Tailwind CSS to deliver a vibrant, highly responsive, and mobile-first user interface. The backend relies on Node.js and Express.js to manage fast RESTful APIs and secure JWT authentication. Real-time buyer-seller messaging is powered by Socket.io, while MongoDB Atlas provides flexible, cloud-hosted NoSQL data storage for user profiles, complex product listings, and chat histories.

4.2.2 About the Database

MongoDB was selected as the core database management system due to its massive flexibility and document-oriented NoSQL structure. By storing information in JSON-like documents, complex data management becomes highly efficient and scalable.

The database contains specific collections critical to e-commerce operations: Users, Products, Messages, and Wishlists. These collections are intricately connected to support the core functionalities of the digital marketplace, from product discovery to real-time chats.

MongoDB Atlas was utilized for cloud-based database hosting, providing secure, highly available, and reliable remote data storage for the university community.

Figure 4.2.2: MongoDB Collections

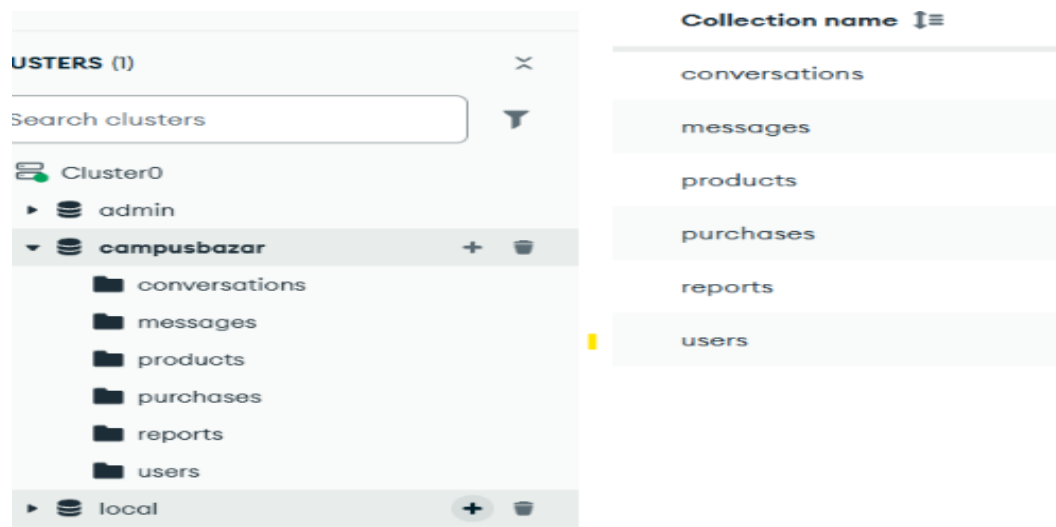
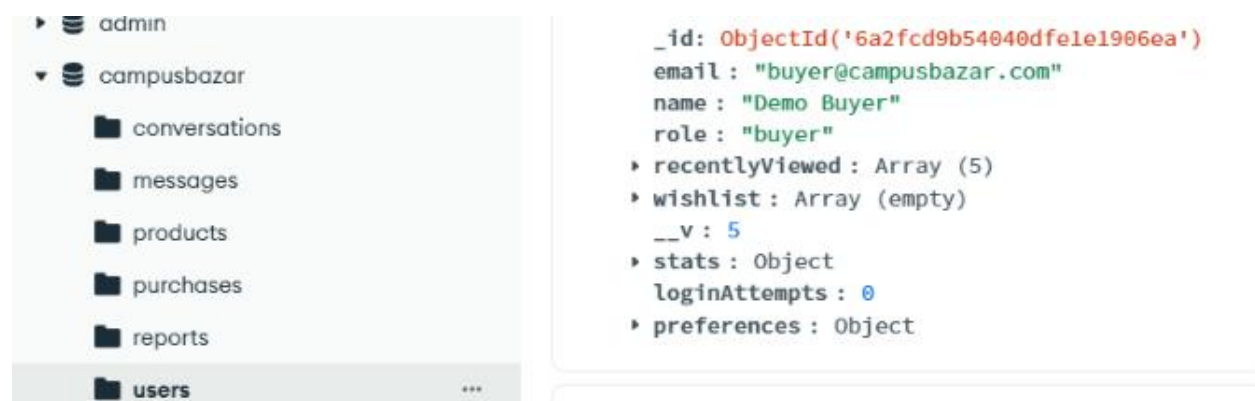


Figure 4.2.3: User Database



4.3 TESTING

4.3.1 INTRODUCTION

The project currently relies on Manual Functional Testing and Manual Integration Testing.

- Manual because there are no automated scripts (like Jest or Selenium) running the tests; a human clicks through the interface to verify it works.
- Functional because the tests are designed to ensure specific features (like logging in or adding products) perform their intended functions.
- Integration because the tests confirm that the React frontend successfully communicates with the Node.js backend and the MongoDB database.

4.3.2: Test Cases Table

Figure: 4.3.2: Test Cases Table

Test ID	Module	Test Scenario	Pre-conditions	Test Steps	Expected Result	Status
TC-01	Authentication	Successful User Registration	User is on the Register page	Enter valid Name, Email, Password, and Role (Buyer/Seller); Click Register	User is successfully registered, saved to the database, and redirected to the login/dashboard page	Pass
TC-02	Authentication	Registration with Existing Email	An account with the email already exists in MongoDB	Enter an already registered email; Fill other fields; Click Register	System shows an error message ("Email already in use") and prevents registration	Pass
TC-03	Authentication	Successful User Login	User has a valid registered account	Enter valid Email and Password; Click Login	User is authenticated, receives a token, and is redirected to their respective Dashboard	Pass
TC-04	Authentication	Login with Invalid Credentials	User is on the Login page	Enter an unregistered email or wrong password; Click Login	System denies access and displays an "Invalid credentials" error	Pass
TC-05	Marketplace	Browse Product Listings	User is logged in as a Buyer	Navigate to the Home/Marketplace page	All active products are fetched from the backend and displayed with images, titles, and prices	Pass
TC-06	Marketplace	Search and Filter Products	Products exist in the database	Enter a keyword in the search bar; Select a category filter	Product list updates dynamically to show only items matching the search and category	Pass

TC-07	Seller Ops	Add a New Product	User is logged in as a Seller	Dashboard; Fill product details; Upload image; Click Submit	Product is added to MongoDB and appears in Marketplace	Pass
TC-08	Seller Ops	Add Product Missing Fields	User is logged in as a Seller	Leave Price or Title empty; Click Submit	Form validation prevents submission and highlights missing fields	Pass
TC-09	Buyer Ops	Add Product to Wishlist	User is logged in as a Buyer	Browse marketplace; Click Add to Wishlist icon	Product is added to wishlist and icon updates visually	Pass
TC-10	Messaging	Real-time Chat Connection	Buyer and Seller are online	Buyer sends a message to Seller through chat	Message is instantly transmitted via Socket.io and appears without page refresh	Pass
TC-11	Admin Ops	View Reported Items	User is logged in as Admin	Navigate to Admin Dashboard → Reports	All reported items are fetched and displayed in a table	Pass
TC-12	Admin Ops	Resolve a Report	User is logged in as Admin	Select an active report; Click Resolve	Report status is updated to Resolved and reflected in UI	Pass
TC-13	Backend Sync	Handle Concurrent Default User Sync	Server starts while frontend hits /api/users/sync concurrently	Start frontend; Start backend immediately after	Backend handles duplicate inserts gracefully without crashing	Fail (Corrected)
TC-14	Process Management	Restart Backend without Port Conflict	Backend crashes or exits and is restarted on same port	Kill background task; Restart backend on port 5000	New instance starts successfully without EADDRINUSE error	Fail (Corrected)

4.4 System Maintenance and Post-Deployment Operations

Post-deployment system maintenance is absolutely critical for the longevity and security of the application. The system is not abandoned once uploaded. Maintenance protocols involve:

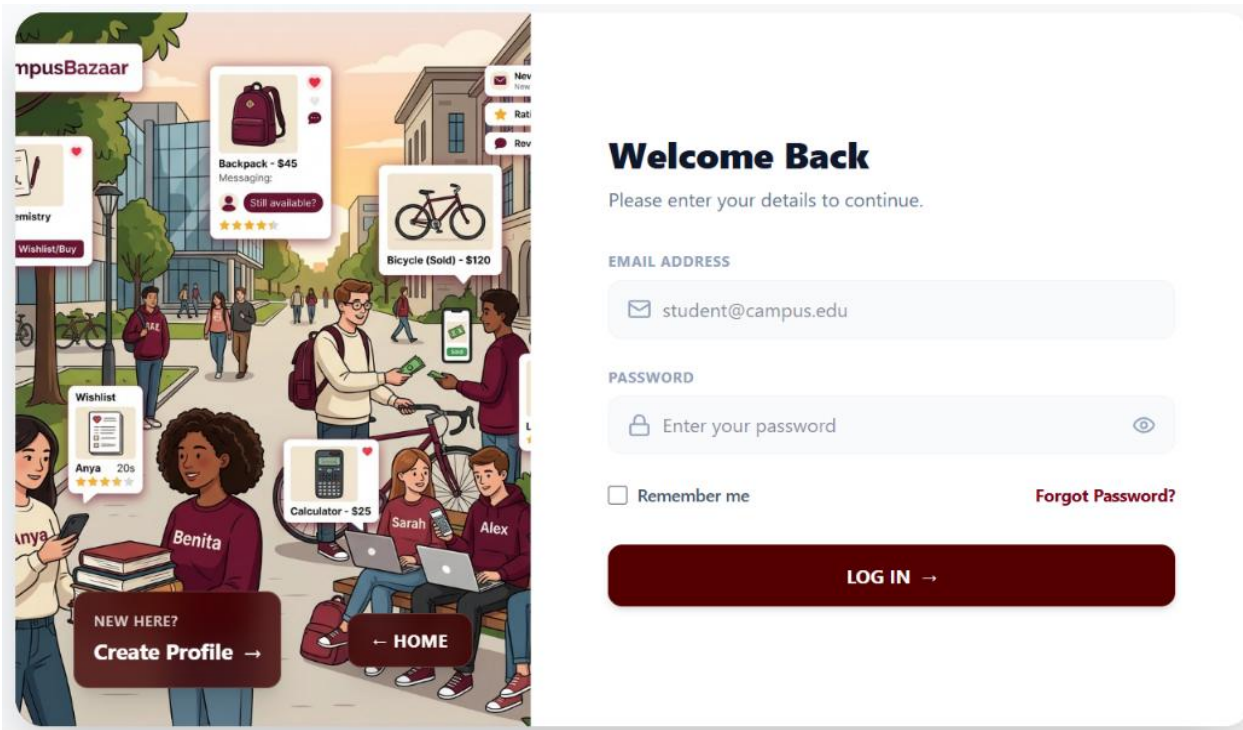
- **Continuous Log Monitoring:** Continuously monitoring server logs via the Render dashboard to catch, diagnose, and patch unhandled runtime errors or memory leaks in the Node.js environment.
- **Security Dependency Updates:** The NPM (Node Package Manager) ecosystem moves rapidly. Maintenance involves regularly running the `npm audit` command and updating package dependencies to patch newly discovered security vulnerabilities (Zero-Day exploits) in open-source libraries.
- **Database Scaling:** Monitoring MongoDB Atlas metrics (read/write operations per second, overall storage size) and upgrading the cluster computing tier automatically if the user base grows exponentially during the chaotic back-to-school season.

Chapter -5

A Session with Software

5.1 The Secure Login Portal

Figure 5.1.1: Login Portal



- Screen Name: User Authentication Portal
- Architectural Purpose: To securely authenticate returning students, establish a secure session via JWT, and grant authorized access to their specific personal data.
- Features and UI Analysis: The UI features a clean, distraction-free minimalist design. It contains strictly validated Email and Password input fields. Crucially, it includes a "Forgot Password" recovery link for accessibility, and a highly prominent, deeply integrated "Sign in with Google" OAuth 2.0 button, allowing students to bypass typing passwords entirely for one-click access.

5.2 The Student Registration Interface

Figure 5.2.1: Student Registration

Create Account
Join the CampusBazar student community.

FULL NAME ROLL NUMBER

EMAIL ADDRESS

MOBILE NUMBER

PASSWORD

- Screen Name: Student Onboarding Form
- Architectural Purpose: To securely onboard new students into the system while validating their credentials against strict database schemas.
- Features and UI Analysis: A comprehensive, visually appealing form requesting the user's Full Name, University Email Address (which features client-side Regex validation to ensure it belongs to an institutional domain), Password, and Password Confirmation fields. It includes

real-time, client-side strength indicators to force users to create secure passwords before hitting the server.

5.3 The Contextual Role Selection Page

Figure 5.3.1: Role Selection (Buyer)

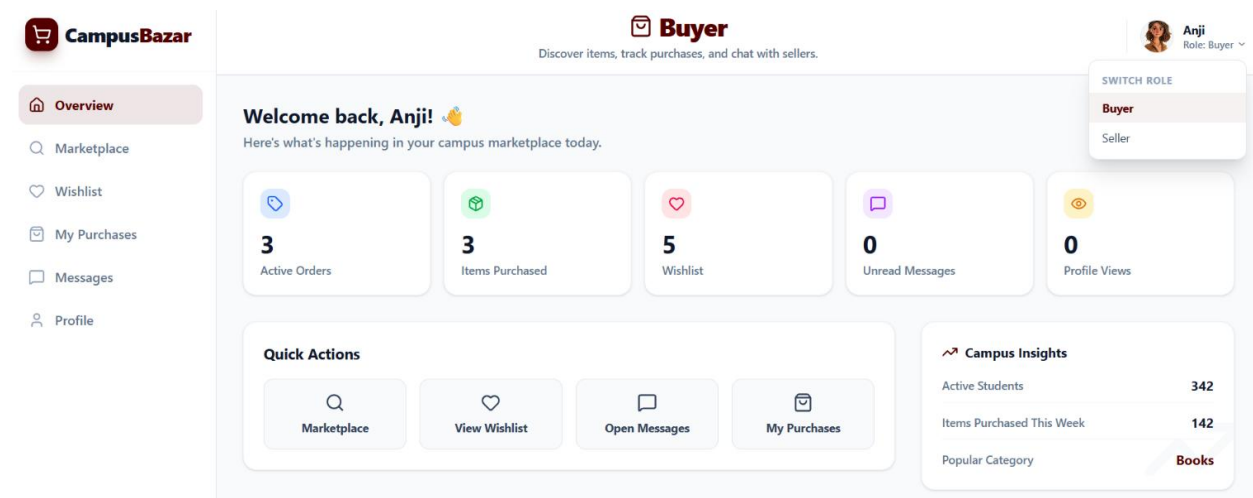
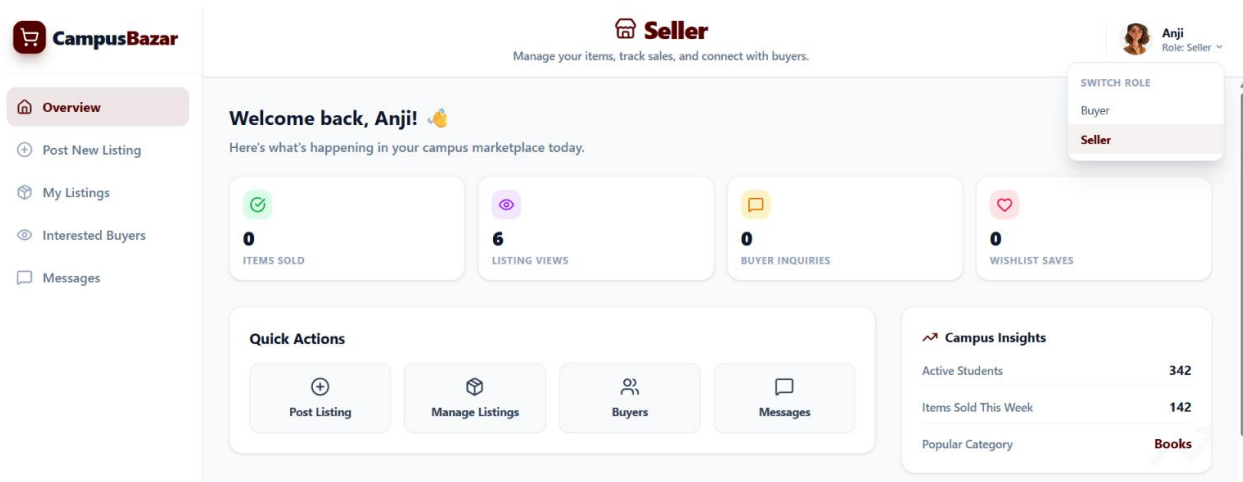


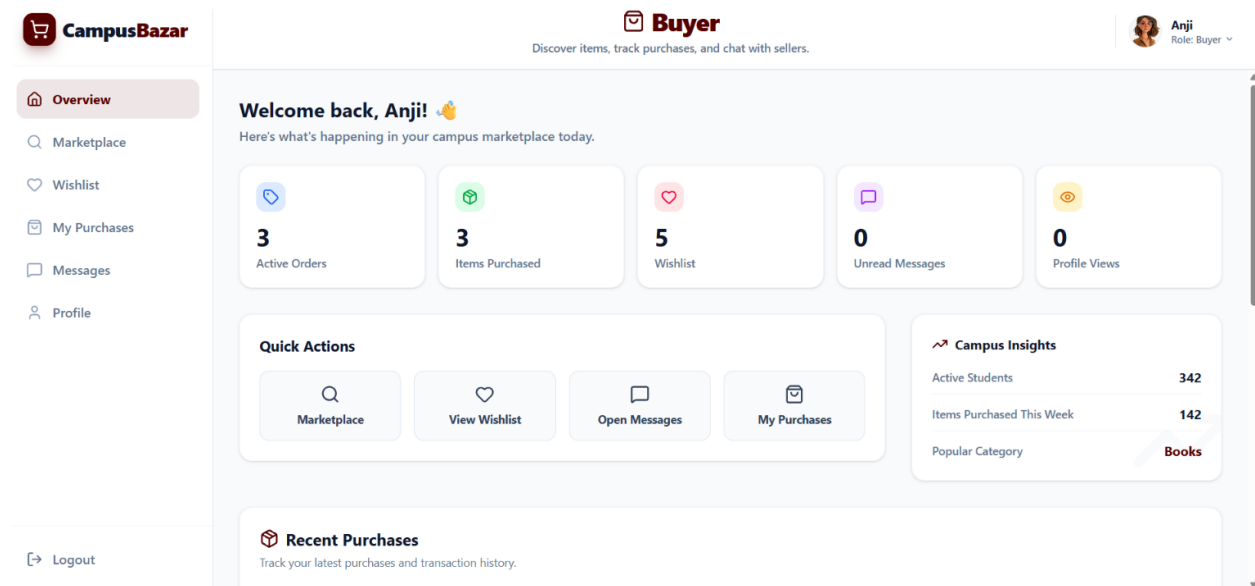
Figure 5.3.2: Role Selection (Seller)



- Screen Name: Context Switcher
- Architectural Purpose: To dynamically tailor the initial user experience based on the student's immediate economic goal upon logging in.
- Features and UI Analysis: Featuring a large, highly visual dual-toggle prompt asking if the user primarily intends to "Buy Items" or "Sell Items". Selecting an option triggers the React Router to instantly load either the consumer-focused Buyer Dashboard or the analytics-focused Seller Dashboard, eliminating unnecessary navigation clicks.

5.4 The Buyer Dashboard (Main Discovery Feed)

Figure 5.4.1: Buyer Dashboard Page



- Screen Name: Main Marketplace Discovery Feed
- Architectural Purpose: Acts as the primary landing page, engineered specifically for maximum item discovery and user retention.
- Features and UI Analysis: Features a dynamic, full-width algorithmic search bar anchored at the top. Directly below, horizontal scrollable category filter pills (e.g., Electronics, Books, Furniture) allow instant filtering. The main body features a visually appealing "Recently Added" horizontal carousel designed to catch the eye, followed by a highly responsive, infinite-scrolling CSS grid of <ProductCard> components showcasing high-quality item images, bold titles, and clear prices.

5.5 The Seller Dashboard (Inventory Hub)

Figure 5.5.1: Seller Dashboard (Overview)

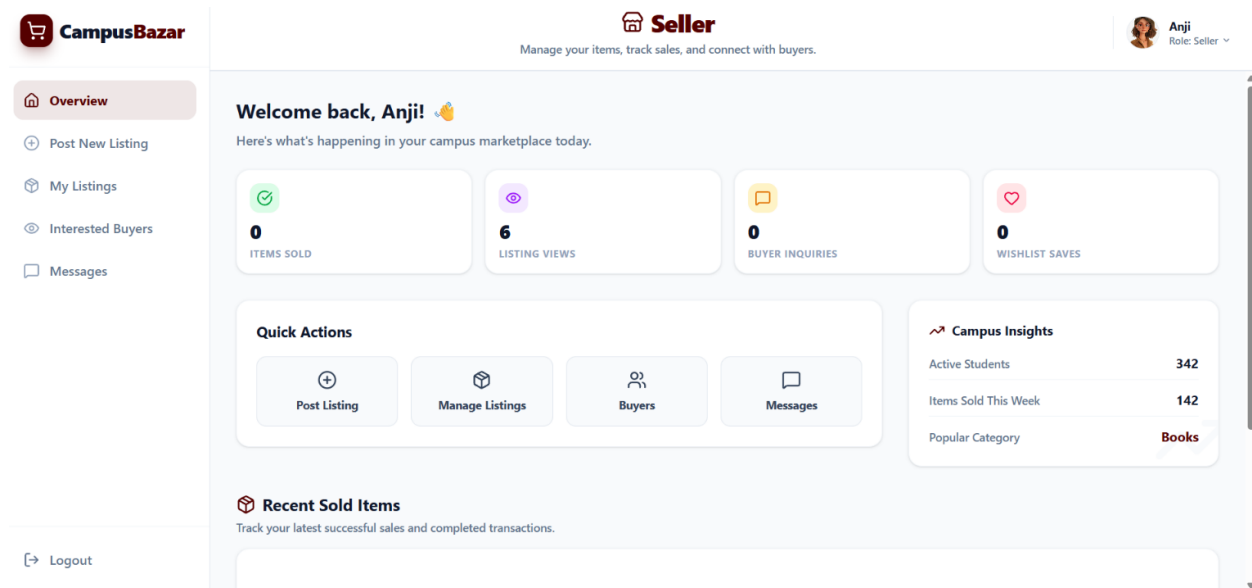


Figure 5.5.2: Seller Dashboard (Post Item)

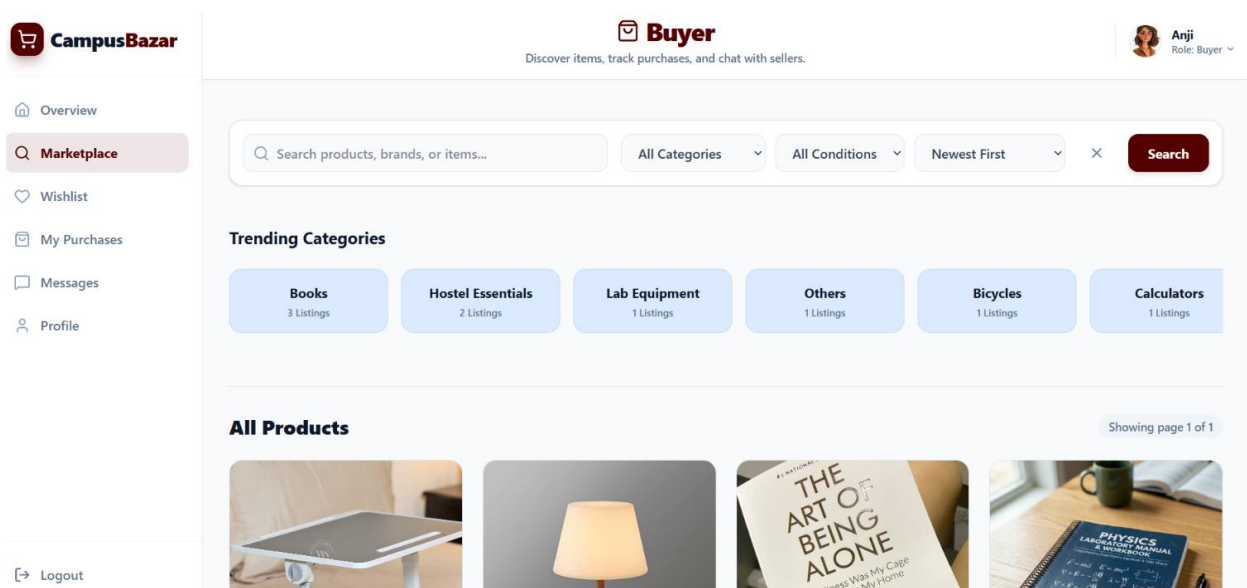
The screenshot displays the 'Post New Listing' form for a user named Anji. The form is divided into several sections:

- Header:** Includes the CampusBazar logo, the user's role 'Seller', and a navigation bar with 'Overview', 'Post New Listing', 'My Listings', 'Interested Buyers', and 'Messages'.
- Category:** A dropdown menu with 'Books' selected.
- Department (Optional):** A text input field with 'e.g., Computer Science'.
- Description:** A text area for describing the condition, features, and reason for selling.
- Pricing:** A section for setting the selling price (₹) and original price (₹) (Optional). The selling price is set to 500, and the original price is set to 1200. A toggle switch for 'Price is negotiable' is also present.
- Condition:** A section for selecting the item's condition: 'New', 'Like New', 'Good' (selected), and 'Fair'.
- Product Title:** A section for entering the product title, with a placeholder 'Product Title Will Appear Here' and a price tag '₹0'.
- Seller Tips:** A section providing tips for sellers, such as using clear photos and setting a competitive price.

- Screen Name: Seller Inventory Management Hub
- Architectural Purpose: Serves as a centralized, data-rich hub for sellers to manage their personal business, track sales, and update inventory.
- Features and UI Analysis: Features top-level analytical metric cards showing "Total Active Listings," "Items Sold," and estimated "Total Earnings." A highly prominent primary call-to-action button invites the user to "Add New Product." Below the metrics, a detailed, paginated data table lists all active inventory, providing quick-action icons to Edit the description or Delete the listing entirely from the database.

5.6 Advanced Browse & Filter Interface

Figure 5.6.1: Browse Items

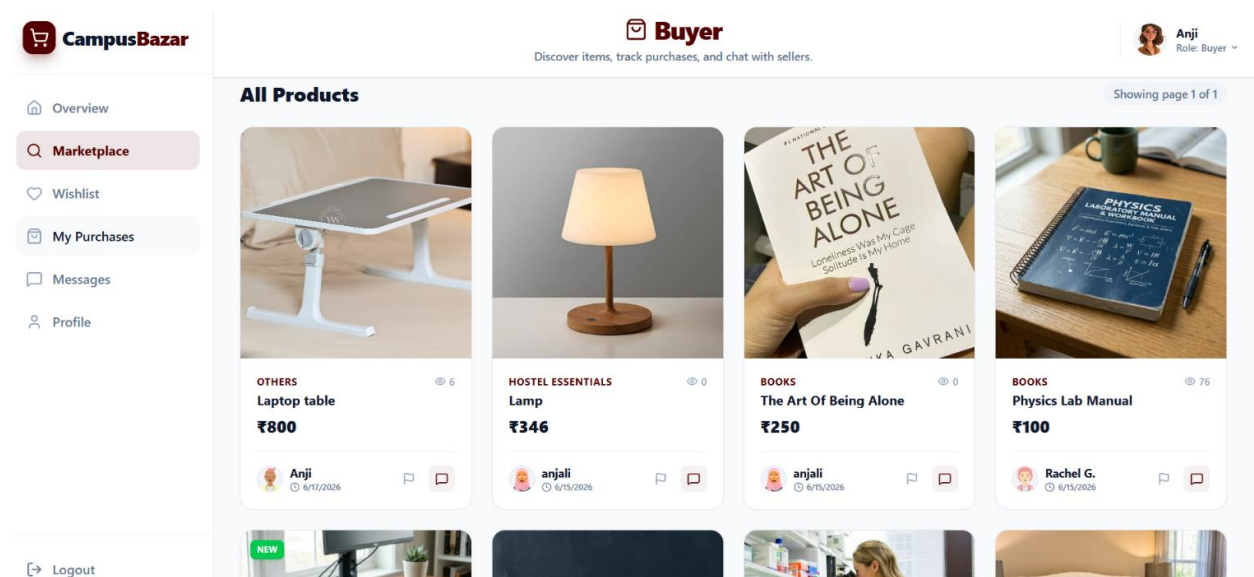


- Screen Name: Advanced Search & Filter Results

- **Architectural Purpose:** Facilitates deep searching and granular filtering for users attempting to locate very specific, niche academic items among thousands of listings.
- **Features and UI Analysis:** A comprehensive left-hand side-panel containing advanced input filters: sliding range inputs for Price (Min/Max), Checkboxes for specific item Conditions (New, Used, Refurbished), and nested Category dropdowns. The top right features sorting options triggering database sorts (Price: Low to High, Newest First). The React state updates the product grid instantly as filters are adjusted without requiring a full page reload.

5.7 Comprehensive Product Details View

Figure 5.7.1: Product Details View

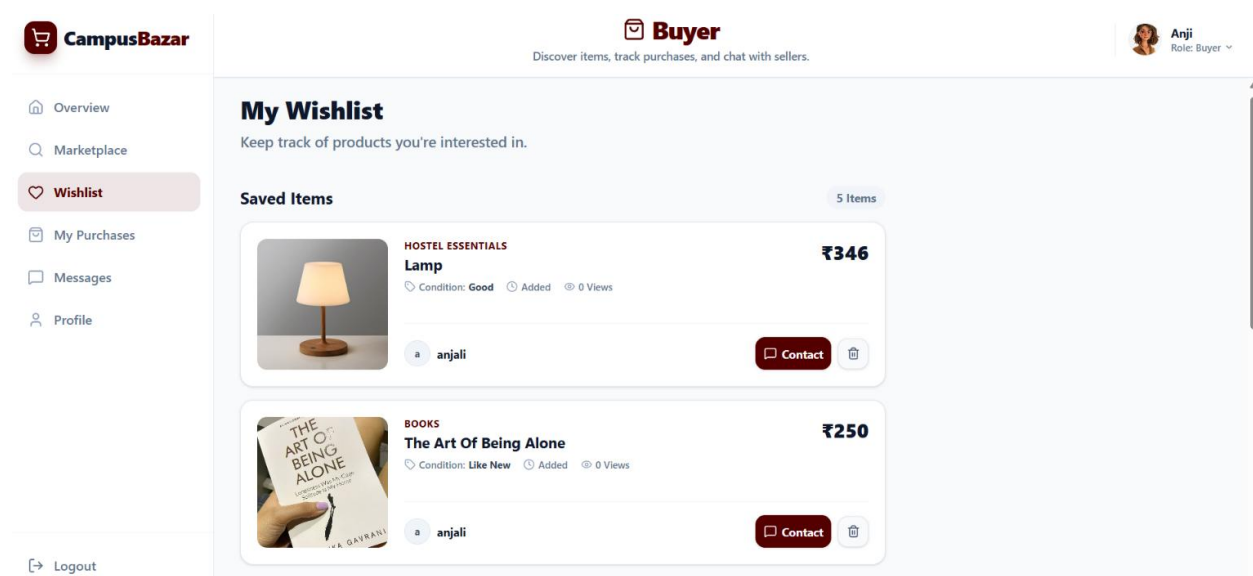


- **Screen Name:** Individual Item View
- **Architectural Purpose:** To display exhaustive, detailed information about a specific item, providing the buyer with all necessary context to make an informed purchasing decision.

- **Features and UI Analysis:** Dominated by a large, interactive image gallery allowing users to swipe through multiple high-resolution photos of the item. Features the item title and price rendered in large, bold typography, followed by a detailed text description block. To the side, a summary card displays the Seller's profile (name, avatar, and join date to establish trust). The page is anchored by two massive call-to-action buttons: a secondary "Add to Wishlist" button and a primary, highly visible "Message Seller" button.

5.8 The Personal Wishlist Repository

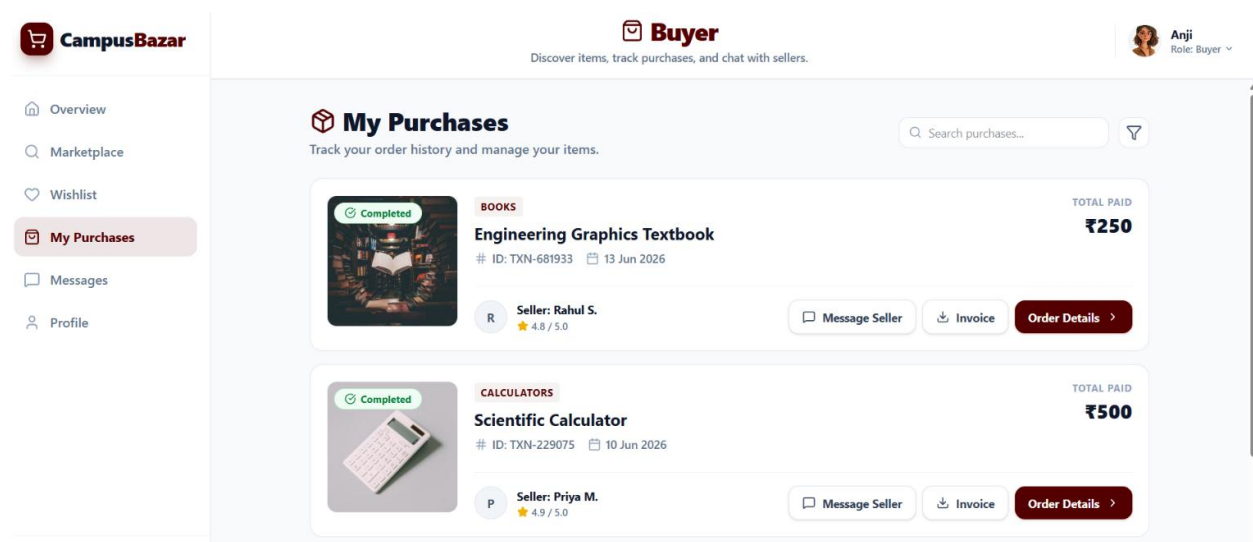
Figure 5.8.1: Wishlist Repository



- Screen Name: Saved Items Repository
- Architectural Purpose: To track and persist items a buyer is interested in evaluating, but is not yet ready to commit to purchasing or initiating a chat for.
- Features and UI Analysis: A clean, uncluttered grid of saved product cards, acting as a personal bookmark system. This allows the user to quickly navigate back to items they are considering. It includes a frictionless, one-click "Remove from Wishlist" button on each card to manage the list easily.

5.9 Historical Purchases Log

Figure 5.9.1: Purchase Log

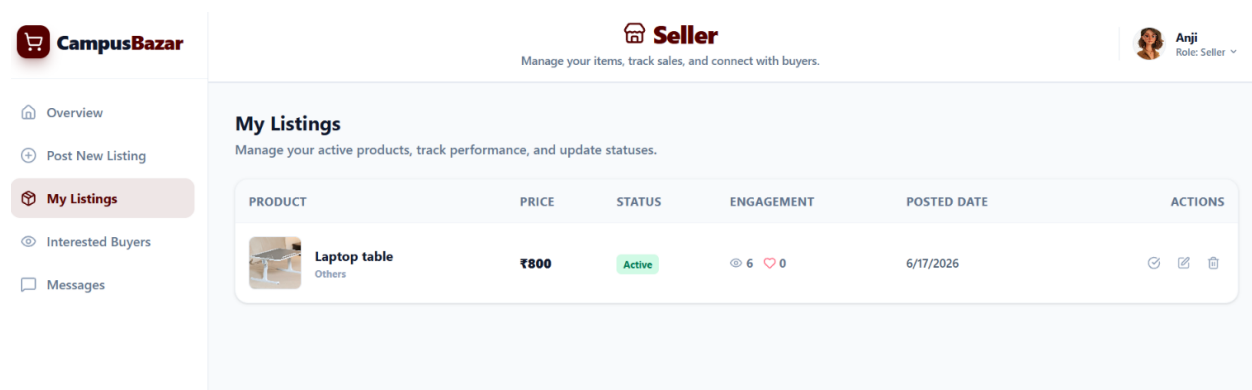


- Screen Name: Historical Transaction Log

- **Architectural Purpose:** Provides a historical, read-only log of all buying activity for personal record keeping and potential dispute resolution.
- **Features and UI Analysis:** A chronologically sorted, tabular list showing items the user has successfully acquired over their time on the platform. Displays the item thumbnail image, the exact timestamp of the transaction, the registered name of the seller, and the final agreed-upon price.

5.10 Active Listings Management

Figure 5.10.1: Listings Management

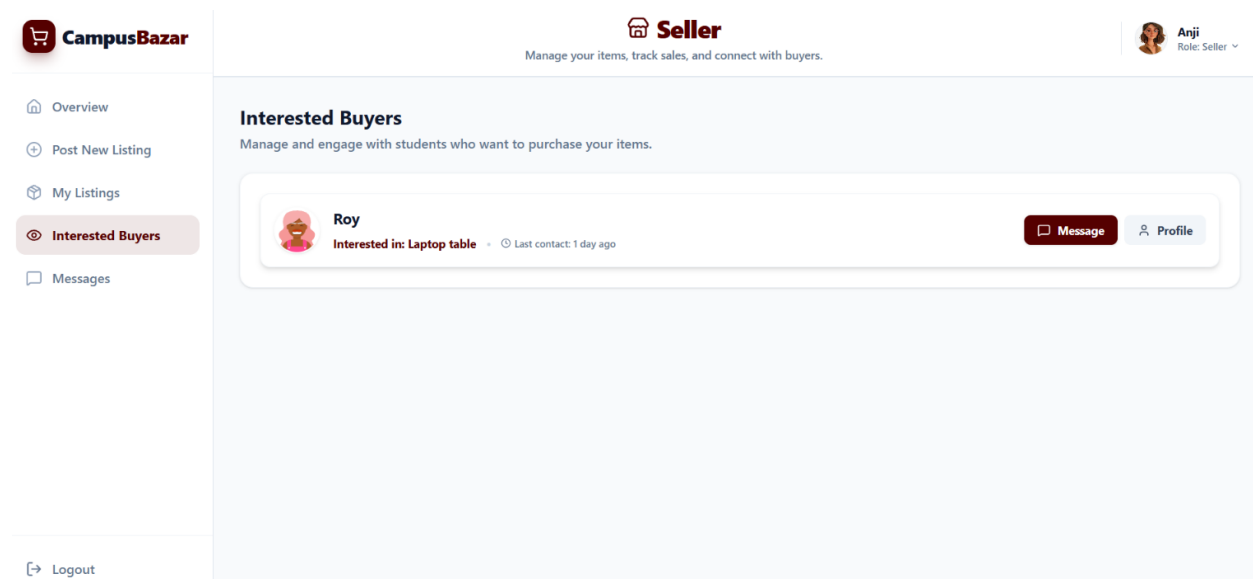


- **Screen Name:** Detailed Inventory View
- **Architectural Purpose:** Provides an expanded, detailed view of a seller's active items, allowing rapid status updates.
- **Features and UI Analysis:** Clean data tables showing active items. Crucially, it includes an interactive toggle switch element allowing the seller to instantly mutate the database status,

marking an item as "Sold". This action immediately and globally removes the item from the public search index to prevent further inquiries.

5.11 Seller Notification Center (Interested Buyers)

Figure 5.11.1: Interested Buyers Page



- Screen Name: Seller Notification Center
- Architectural Purpose: To rapidly alert sellers of potential sales and incoming chat inquiries, minimizing response times.

- Features and UI Analysis: A dedicated notification panel visually highlighting which specific users have initiated a new chat regarding specific products. Clicking a specific notification card drops the seller directly into the isolated chat room with that buyer.

5.12 Real-Time WebSocket Messaging Module

Figure 5.12.1: Message From Seller Page

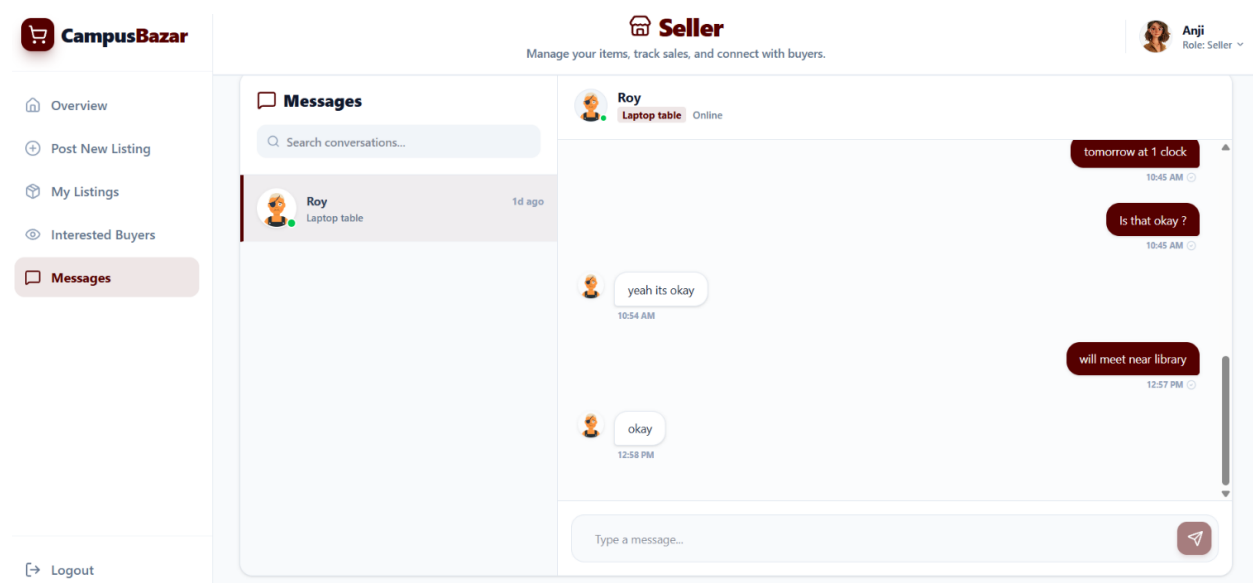
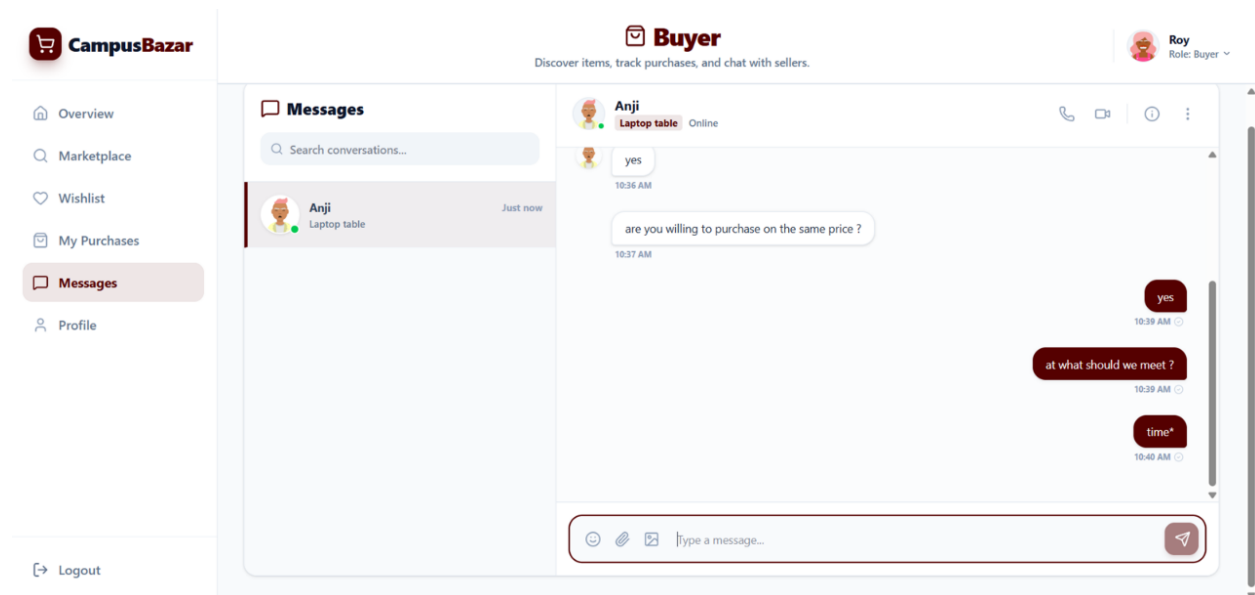


Figure 5.12.2: Message From Buyer Page



- Screen Name: Real-Time Chat Interface
- Architectural Purpose: The crown jewel of the platform; designed to facilitate secure, instant, real-time peer-to-peer communication and price negotiation without page reloads.
- Features and UI Analysis: A highly sophisticated split-screen interface mimicking modern chat applications like WhatsApp or iMessage. The left pane shows a scrollable list of all active, historical chat threads. The right pane displays the active real-time message thread bubbles. It includes visual blue-tick read receipts, exact message timestamps, and a sticky header indicating the exact product the chat is referencing to prevent confusion.

5.13 User Settings and Preferences

Figure 5.13.1: Settings Page

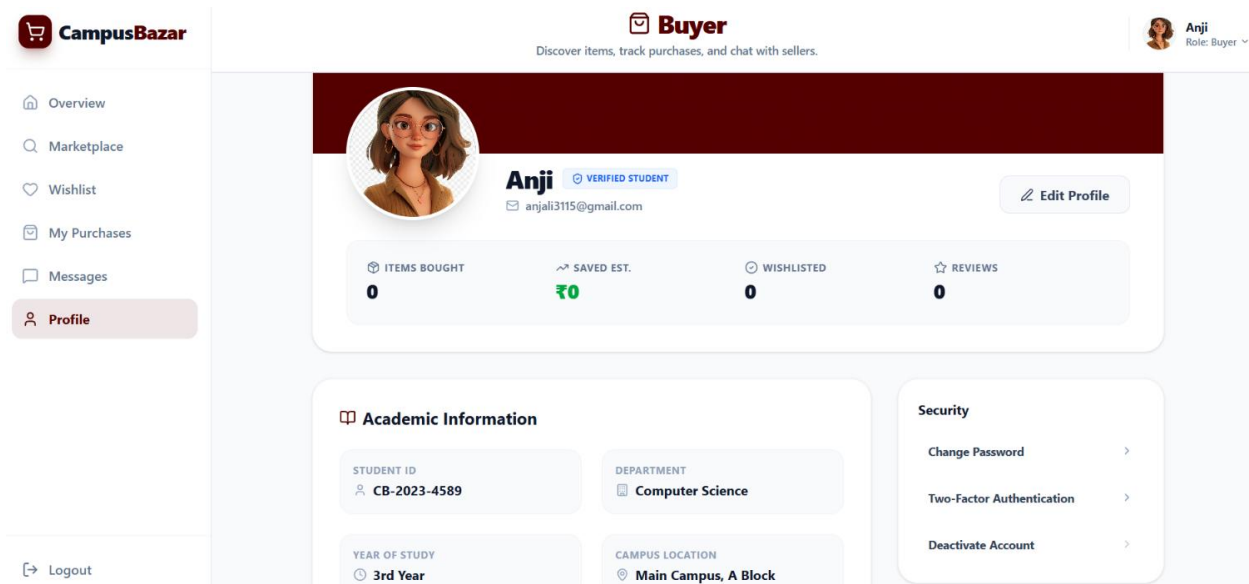
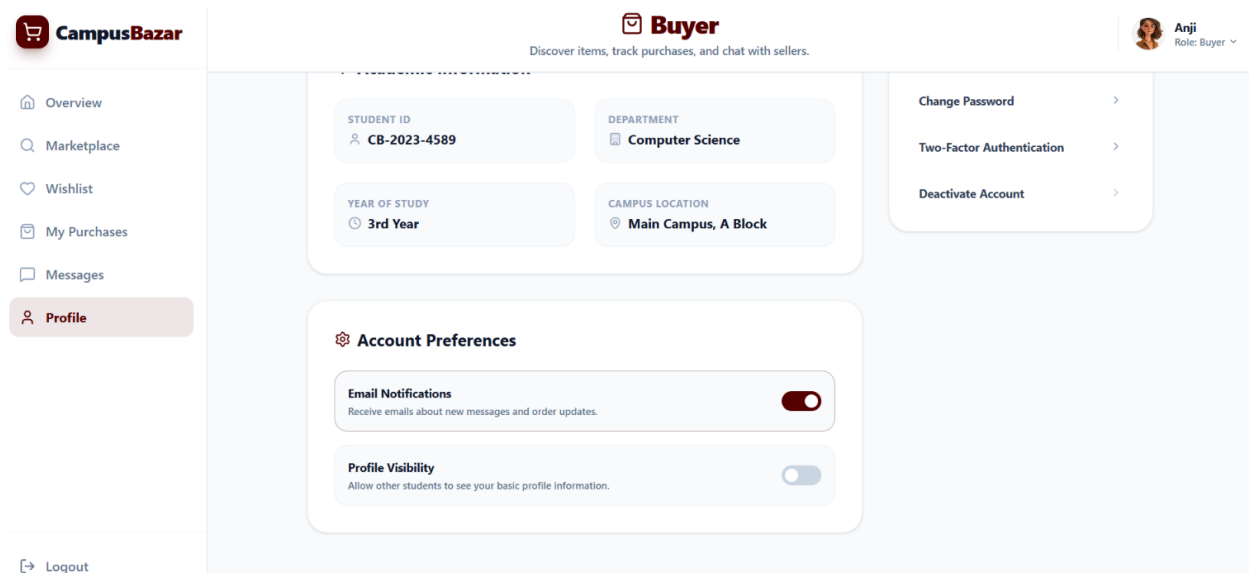


Figure 5.13.2: Account Preferences

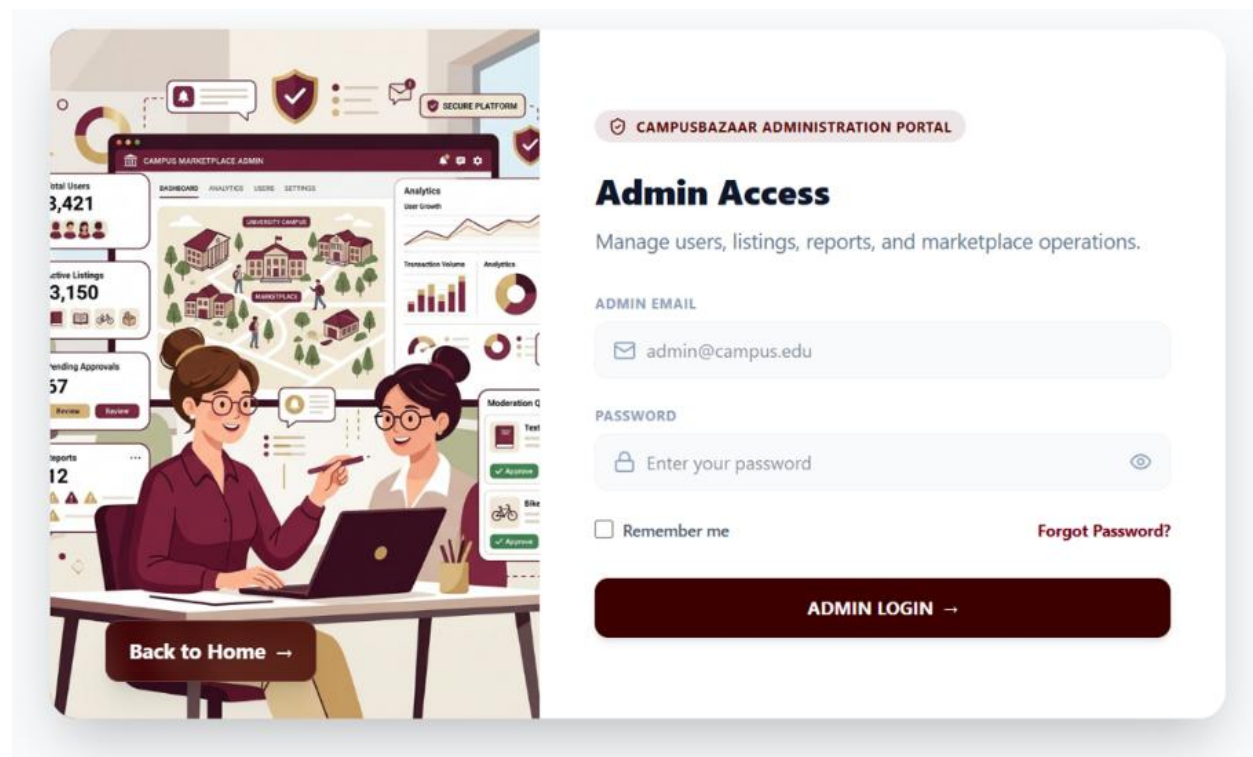


- Screen Name: User Preferences

- Architectural Purpose: To allow users to customize their account, manage security, and personalize the application interface.
- Features and UI Analysis: Intuitive forms allowing the user to securely upload and update their profile picture avatar, change their encrypted password, toggle the global UI theme between Light mode and Dark mode, and manage their email notification settings.

5.13 Secure Administrator Login Portal

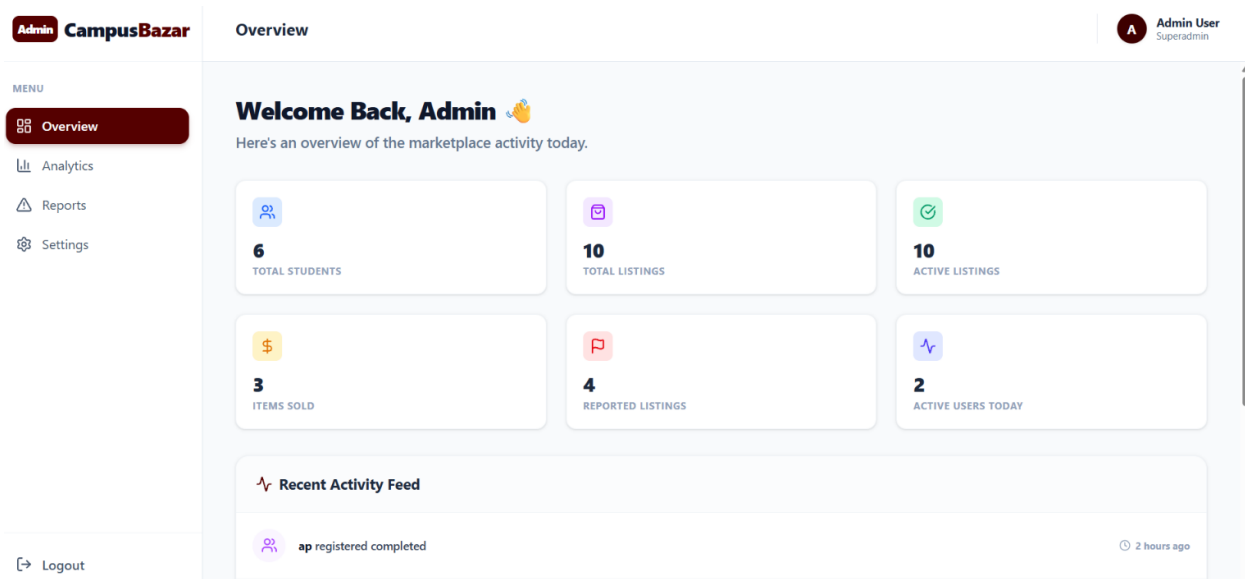
Figure 5.13.1: Admin Login Portal



- Screen Name: Secure Administrator Portal
- Architectural Purpose: A highly secure, mathematically hidden login portal strictly reserved for platform administrators.
- Features and UI Analysis: A high-security, utilitarian login form that intentionally bypasses standard user routing and public navigation links. Accessing this requires specific URL knowledge and admin-flagged database credentials.

5.14 Global Administrator Dashboard

Figure 5.14.1: Admin Dashboard

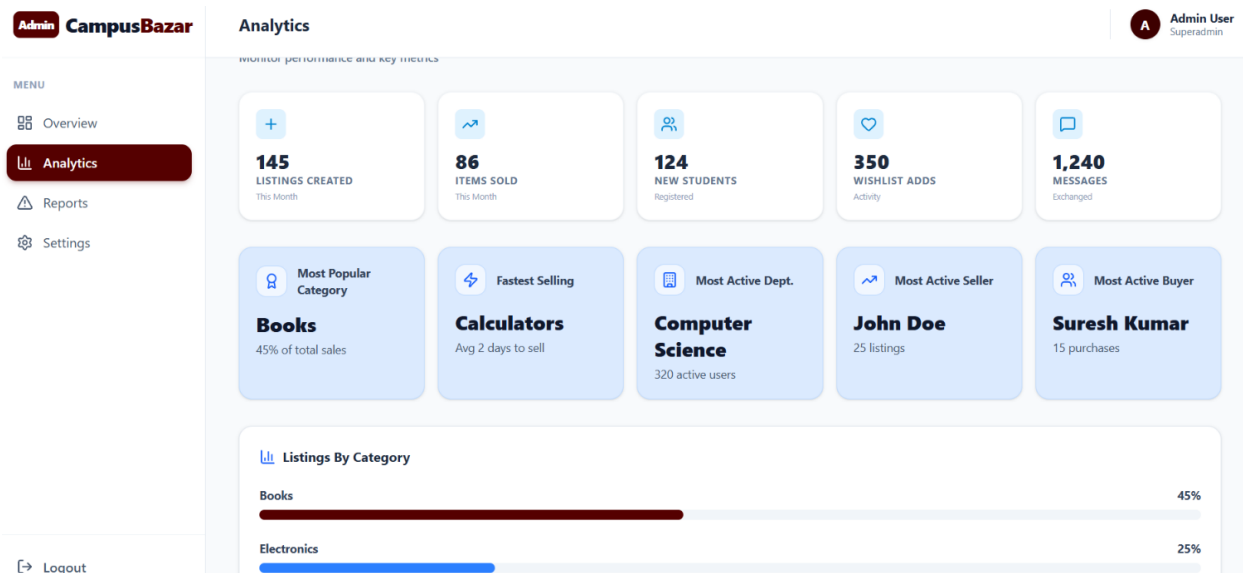


- Screen Name: Global System Overview

- Architectural Purpose: To provide authorized administrators with a macro-level, bird's-eye view of the overall health, traffic, and activity of the entire platform.
- Features and UI Analysis: Dominated by large, colorful statistical cards showing live counts of Total Registered Users, Active Listings currently in the database, and a live-feed widget of new users joining the platform.

5.15 System Analytics and Data Visualization

Figure 5.15.1: Analytics Page(Admin)



- Screen Name: Data Visualization Center

- Architectural Purpose: To visually interpret complex marketplace data trends for administrative reporting and future system planning.
- Features and UI Analysis: Beautifully rendered, interactive charts (utilizing React charting libraries). Includes bar charts showing the most heavily trafficked product categories and line graphs displaying user registration growth over the course of the academic semester.

5.16 Moderation and Reports Queue

Figure 5.16.1: Reporting From User Dashboard

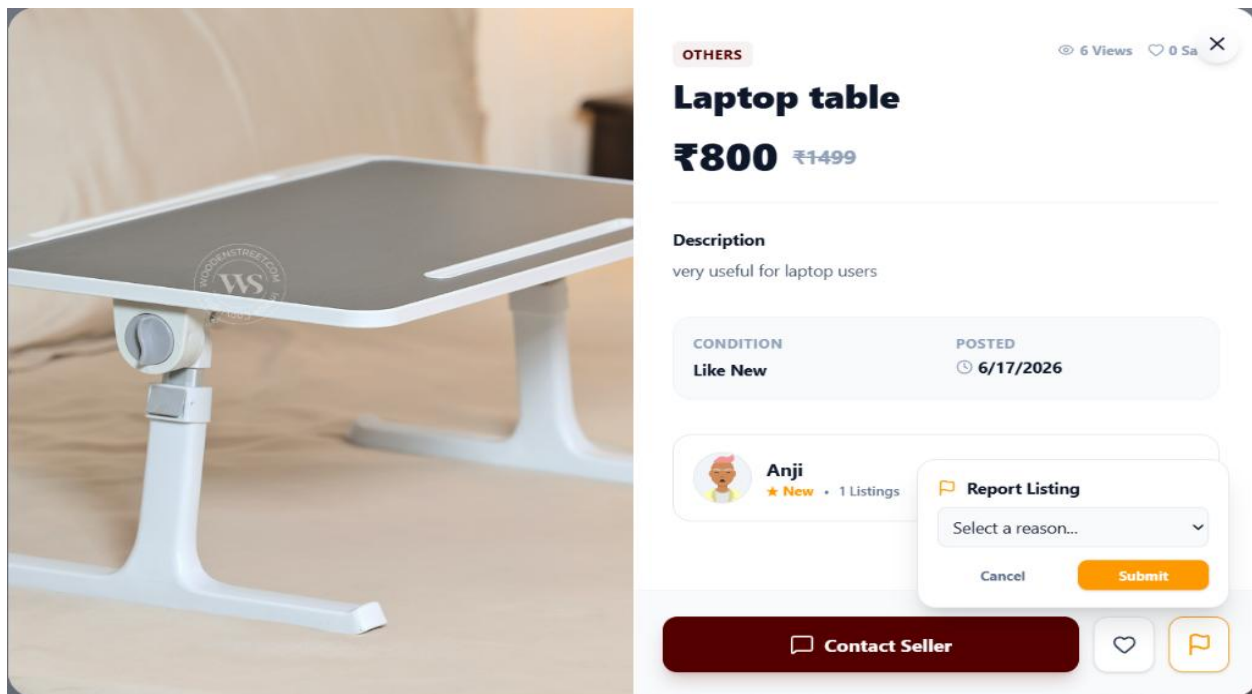
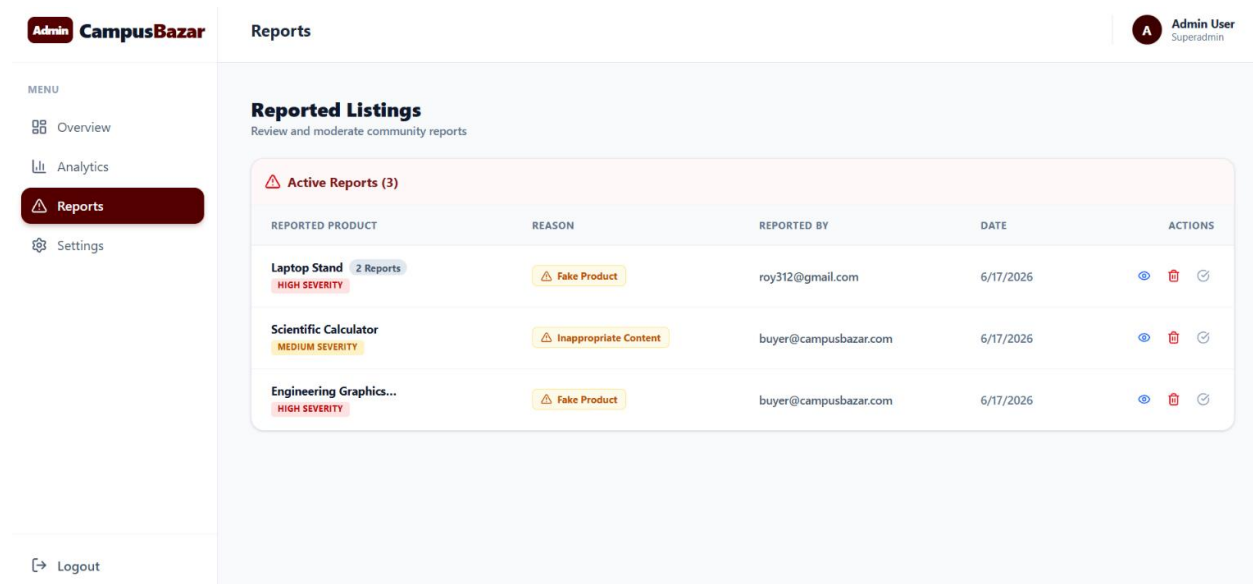


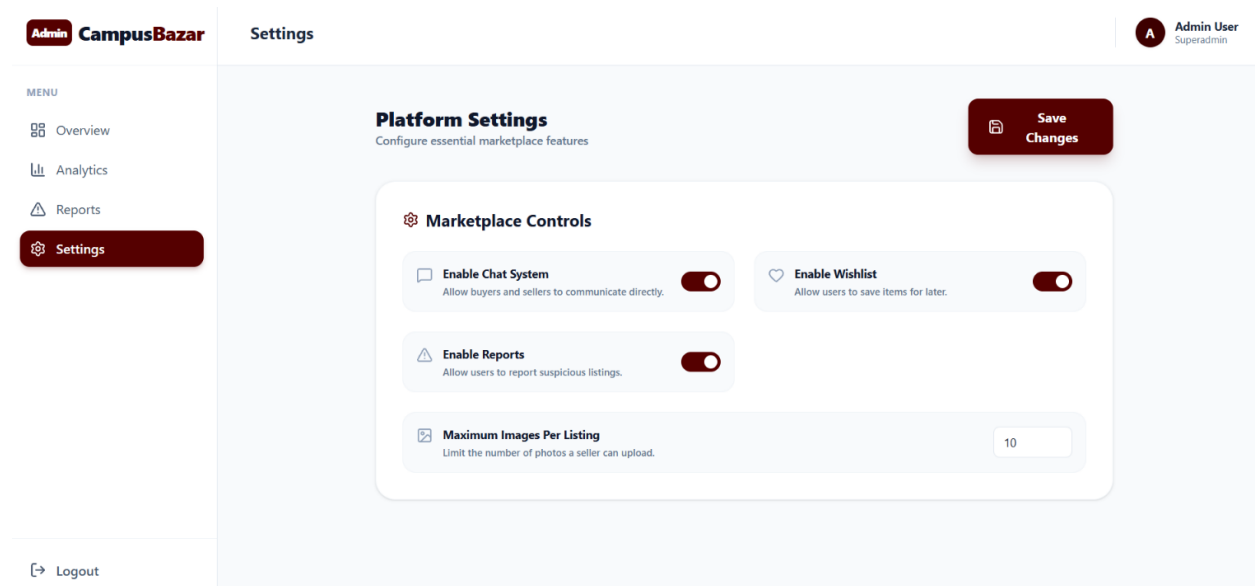
Figure 5.16.1: Accepting Report By User



- Screen Name: Community Moderation Queue
- Architectural Purpose: To enforce and maintain strict community safety guidelines and process user-submitted complaints efficiently.
- Features and UI Analysis: A robust data table listing all user-submitted flags regarding inappropriate listings or abusive chat behavior. The admin interface provides immediate, powerful action buttons to "Delete Post," "Dismiss Report," or permanently "Ban User."

5.17 Admin Settings

Figure 5.17.1: Admin Settings Page



- Screen Name: Admin Settings
- Architectural Purpose: To allow users to customize their account, manage security, and personalize the application interface.
- Features and UI Analysis: Intuitive forms allowing the user to securely upload by enabling the chat system, enabling reports and enabling the Wishlist.

Chapter -6

Conclusion

6.1 Conclusion

The CampusBazaar software engineering project successfully demonstrates the comprehensive research, architectural design, full-stack development, and cloud deployment of a hyper-localized digital marketplace. This platform was specifically tailored to address the unique economic dynamics and safety requirements of a university student community. By strategically abandoning rigid traditional development models and instead leveraging the Agile software development methodology alongside the highly robust, scalable MERN stack (MongoDB, Express, React, Node.js), the platform definitively solves critical, real-world issues. It eradicates the safety risks of meeting strangers, protects user privacy by eliminating the need to share phone numbers, and solves the organizational inefficiency that heavily plagues traditional, manual student commerce methods like paper flyers.

The successful, bug-free implementation of highly complex, modern web features—such as the real-time bidirectional WebSocket messaging engine, secure stateless JWT-based authentication, and highly intuitive, responsive, role-based dynamic React dashboards—provides an incredibly seamless, professional, and "app-like" user experience. Ultimately, CampusBazaar achieves its primary, overarching socio-economic goal: empowering college students to save significant amounts of money by bypassing monopolies, efficiently recycling perfectly good goods, and interacting safely within a secure, closed-loop academic digital ecosystem. The project serves as a prime, functional example of how modern web technologies can be rapidly utilized to solve specific, localized community problems effectively.

6.2 Future Enhancements and Scalability

While the currently deployed version of the CampusBazaar system is fully functional, highly robust, and meets all baseline academic and operational objectives, modern software is never truly "finished." Several advanced, high-level enhancements have been identified through user feedback. These features can be systematically integrated in future Agile development iterations to elevate the platform from a campus project to a commercial-grade enterprise application:

- **Online Escrow Payment Gateway Integration:** Integrating advanced, secure payment processors like Stripe API or Razorpay to allow users to pay for items online via debit/credit cards. Crucially, the system would hold the funds securely in "escrow" until the buyer physically inspects and explicitly accepts the item in person. This adds an immense layer of financial security, prevents physical cash robberies, and guarantees the seller gets paid.
- **AI-Driven Product Recommendation System:** Implementing sophisticated machine learning algorithms (such as collaborative filtering) on the backend to analyze a specific buyer's historical search queries, previous purchases, and wishlist data. The system would then automatically suggest highly relevant, personalized items on their dashboard, significantly increasing the platform's overall sales conversion rate.
- **Computer Vision and AI-based Visual Product Search:** Utilizing advanced AI image recognition APIs (such as Google Cloud Vision) to allow users to simply take a picture of an item from their smartphone camera (like a specific textbook cover) and have the system automatically search the database for visual matches, bypassing the need for manual text search entirely.
- **Real-Time Push Notifications and SMS:** Integrating Progressive Web App (PWA) Push Notification service workers or SMS text alerts via the Twilio API. This ensures users are instantly notified of new incoming chat messages, price drops on wishlisted items, or successful sales even when the web application browser tab is completely closed.
- **Native Mobile Application Porting:** Porting the core React.js web application business logic into a native React Native application for direct deployment on the Apple iOS App Store

and Google Play Store. A native app would allow the platform to fully leverage native device hardware, including direct access to the smartphone camera for faster uploads, GPS for mapping safe meetup locations on campus, and native operating system push services.

- Advanced Seller Analytics and Big Data Dashboard: Providing sellers with highly granular, big-data analytics regarding their inventory. This would show sellers exactly how many "impressions," "clicks," and "wishlist adds" their specific products are generating per day on a line graph, empowering them to adjust their pricing strategies or update their photos dynamically to increase their chances of a sale.

6.3 References and Bibliography

1. React Core Development Team. (n.d.). React – A declarative, efficient, and flexible JavaScript library for building user interfaces. Retrieved from <https://reactjs.org/>
2. Node.js Foundation. (n.d.). Node.js v18.x Official API Documentation. Retrieved from <https://nodejs.org/en/docs/>
3. MongoDB Inc. (n.d.). MongoDB Atlas Cloud Database and Mongoose Object Data Modeling (ODM) Documentation. Retrieved from <https://docs.mongodb.com/>
4. Express.js Core API. (n.d.). Fast, unopinionated, minimalist web framework for Node.js. Retrieved from <https://expressjs.com/>
5. Socket.io Development Team. (n.d.). Bidirectional and low-latency, real-time communication for every platform. Retrieved from <https://socket.io/>
6. Fowler, M., Highsmith, J., et al. (2001). The Agile Manifesto for Software Development. *Software Development*, 9(8), 28-35.
7. Sommerville, I. (2015). *Software Engineering: Principles and Practice* (10th ed.). Pearson Education.

8. Tailwind Labs. (n.d.). Tailwind CSS Documentation: Rapidly build modern websites without ever leaving your HTML. Retrieved from <https://tailwindcss.com/docs>
9. Auth0 / JWT.io. (n.d.). JSON Web Tokens (JWT) Introduction and Cryptographic Signing. Retrieved from <https://jwt.io/introduction>
10. MDN Web Docs. (n.d.). WebSocket API - Web APIs. Retrieved from https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API

